

P2P e Wireless



P2P

Client Server

Peer to Peer

Grid

Grid Vs P2P

Cloud

P2P

Livelli del P2P

Tipologie di sistemi P2P

Valutazione di una topologia di rete overlay

Tipi di topologia di rete P2P

Centralizzato

Anello

Gerarchica

Decentralizzata a grafo

Approcci ibridi

Anello con nodi esterni

Centralizzata con decentralizzata

Sistemi non strutturati

Sistemi strutturati

Distributed Hash Tables (DHT)

Protocolli P2P

Napster

Pacchetti

Protocollo

Architettura

Gnutella 0.4

Pacchetti

Regole

Fenomeno del Free riding

Flooding e colli di bottiglia

Gnutella 0.6

Architettura

- Ricerca di un file
- Gnutella 2
 - Architettura
 - Operazione di query
- FastTrack
- OpenFT
- EDonkey
- EMule
 - Identificativi
 - Startup
 - Gestione dei file
 - Query
 - Reputazione per la coda di upload
- BitTorrent
 - Architettura
 - Operazioni
 - Ricerca
 - Pubblicazione
 - Swarm
 - Tit-for-Tat
- Freenet
 - Architettura
 - Storage
 - Identificativi
 - Keyword Signed Key (KSK)
 - Signed Subspace Key (SSK)
 - Content Hash Key (CHK)
 - Routing
 - Inserimento di un file
 - Ricerca di un file
 - Ingresso di nuovi nodi
 - Anonimato
- Chord
 - Funzionalità
 - Identificatori
 - Routing
 - Ricerca
 - Ingresso di nuovi nodi
 - Uscita di un nodo
- Pastry

Routing

- Leaf set

- Neighborhood set

- Routing table

- Ingresso di nuovi nodi

Kademlia

- Identificativi

- Trie

- K-Bucket

- Aggiornamento del k bucket

- Messaggi

- Parallelismo nella ricerca

- Ingresso di nuovi nodi

- Inserimento di un file

Tabelle riassuntive

- Sistemi ibridi: Napster

- Sistemi non strutturati: Gnutella 0.4, Freenet

- Sistemi strutturati: Gnutella 0.6, Gnutella 2, FastTrack, OpenFT, Emule, BitTorrent

- DHT: Chord, Pastry, Kademlia

Wireless

- Cablato Vs Wireless

- Tassonomie

- Livello fisico

- Raggio di ricezione e interferenza

- Tecniche di suddivisione del canale

- Time-Division Multiplexing (TDM)

- Frequency Division Multiplexing (FDM)

- Code Division Multiplexing (CDM)

- Aloha e Slotted-Aloha

- Carrier Sense Multiple Access - Collision Avoidance (CSMA-CA)

- Cellular wireless networks

- GSM

- IEEE 802.11

- Livello fisico

- Spread spectrum signal (SSS)

- Frequency-hopping spread spectrum (FHSS)

- Direct-sequence spread spectrum (DSSS)

- Configurazioni

- Versioni di IEEE 802.11

IEEE 802.11: prevenzione delle collisioni

Distributed Coordinator Function (DCF)

Protocollo MACAW

Tempi di attesa

Frame molto lunghe

CTS-to-self

Compatibilità 802.11b e 802.11g

Compatibilità 802.11n

Throughput

IEEE802.11: Point Coordinator Function (PCF)

IBSS

Formato frame

Protocolli di routing

Approcci di routing

Approccio proattivo

Approccio reattivo

Approcci ibridi

Mobile IP

Reverse tunneling

Routing per reti wireless: approccio reattivo

Dynamic Source Routing (DSR)

Ad hoc Distance Vector (AODV)

Location Aided Routing (LAR)

Link reversal Algorithm (LRA)

Temporally Ordered Routing Algorithm (TORA)

Altri protocolli

Routing per reti wireless: approccio proattivo

Destination Sequenced Distance Vector (DSDV)

Clusterhead Gateway Switch Routing (CGSR)

Optimized Link State Routing (OLSR)

Routing per reti wireless: approccio ibrido

Zone Routing Protocol (ZRP)

Delay Tolerant Network (DTN)

DTN vs MANET

Caratteristiche delle DTN

Bundle layer

Routing

Routing opportunistico

Social based routing

Epidemic

Spray and wait

ProPhet

Varianti TCP

Problemi di TCP

Sistema auto-sincronizzante o ACK clock

Algoritmi TCP

Slow start

Congestion Avoidance

Fast retransmit

Fast recovery

TCP Tahoe

TCP Reno

Algoritmo di Nagle

Retransmission Time Out (RTO)

Fast Retransmit & Fast recovery

Slow start (Tahoe) vs Fast recovery (Reno)

TCP NewReno

Reset del timer

TCP Vegas

Congestion avoidance

Cambio di route

Congestione persistente

Sistema di ritrasmissione

Confronto fra TCP Tahoe, Reno, NewReno e Vegas

Selective ACKnowledgements (SACK)

Data Reneging

TPC Westwood

BandWidth Estimate (BWE)

Gestione della perdita di un pacchetto

TCP Venet

TPC BIC

P2P

I protocolli di tipo Peer to Peer (P2P) negli ultimi anni hanno avuto un esplosione esponenziale, a discapito di altre tipologie di traffico quali web tradizionale, email o ftp. Questo perché un modello P2P ha delle caratteristiche che lo rendono appetibile, soprattutto in presenza di autorità centrali di cui si vuole evitare il controllo e per la comunicazione e condivisione non mediata (e quindi non controllata) fra utenti.

Client Server

Il modello client server è il modello classico che ha presto preso il sopravvento per la comunicazione tramite internet. Si sposa bene con il modello delle socket, che prevede un richiedente e un servente.

I limiti del modello, però, nascono con la **scalabilità** limitata del servizio, che supporta solo un numero limitato di client. Aumentare il numero di server introduce problemi di coerenza di dati e coordinazione. Ci sono anche importanti limiti riguardo la **fault tolerance**, specie se non c'è ridondanza. Le **risorse** non sono ben utilizzate: il client non utilizza le proprie capacità computazionali, delegando tutto al server. In parte il problema è stato ovviato con javascript sul browser web.

In compenso un'architettura server non è particolarmente complicata.

Peer to Peer

Nel modello Peer to Peer tutti i partecipanti hanno le stesse capacità e possono iniziare una sessione di comunicazione. Non ci sono ruoli predefiniti. Al contrario, ogni nodo può svolgere un compito diverso in base alla necessità.

I nodi hanno dei tempi di vita indipendenti, e sono in grado di eseguire operazioni in maniera decentralizzata.

Il DNS può essere visto come un esempio molto semplice di una rete P2P.

Per poter effettuare il routing dei pacchetti, è necessario realizzare una rete virtuale con delle proprie regole di routing. La rete virtuale astrae e nasconde l'utilizzo della rete internet sottostante.

Overlay network

Rete virtuale sopra la rete internet reale. Introduce una gestione di rete differente, con appositi nodi che fanno da router. Per poter entrare nella overlay network potrebbe essere comunque necessario riferirsi ad un nodo specifico su internet.

Il modello P2P deve essere dinamico, poiché ogni nodo deve poter uscire ed aggiungersi alla rete senza causare disservizi. Il sistema deve possedere i seguenti aspetti tecnici:

- **controllo decentralizzato:** non c'è un'autorità centrale che sancisce i movimenti all'interno della rete
- **adattabilità automatica:** l'architettura della rete è soggetta a cambiamenti continui e automatici
- **scalabilità:** l'inserimento di nuovi utenti migliora le prestazioni (o comunque non le peggiora)
- **dinamicità:** l'ingresso o l'uscita degli utenti non deve sconvolgere il sistema e non deve prevedere operazioni particolari
- **resistenza ai guasti:** il guasto o uscita forzata di un nodo non provoca problemi o disservizi alla rete
- **reperibilità delle risorse:** se una risorsa è disponibile, deve essere reperibile
- **efficienza:** i nodi non devono impiegare tutte le proprie risorse per partecipare al sistema
- **economia:** la rete deve minimizzare l'uso delle risorse e preferibilmente limitarsi a sfruttare quelle inutilizzate
- **anonimato:** gli utenti e le loro attività dovrebbero essere anonimi o almeno complessi da tracciare

L'ovvia challenge di tutte le reti P2P è dover gestire il comportamento di tutti i nodi senza la presenza di un coordinatore centrale. Questo vuol dire che tutti i nodi devono essere messi nelle condizioni di conoscere lo stato della rete e di cooperare potendo comunicare solo con altri nodi loro pari.

È necessario che ogni nodo sia in grado di lavorare e contribuire all'intera rete conoscendo solo informazioni locali.

La capacità dei nodi, inoltre, è eterogenea. Bisogna assicurarsi che anche l'ingresso di nodi con capacità ridotte forniscano un servizio alla rete, e non un disservizio.

Churn rate

Frequenza con cui i nodi entrano o escono del sistema P2P

Grid

Il modello grid è uno schema utilizzato soprattutto in ambito di calcolo distribuito. Tali sistemi prevedono che l'enorme mole di lavoro da eseguire venga diviso tra quanti più agenti possibile secondo un'organizzazione coordinata.

Generalmente è presente un controllo centrale che stabilisce la suddivisione dei *job* che ogni nodo periferico dovrà compiere, in maniera coerente con le sue capacità.

Si tratta comunque di reti con dimensioni limitate, con infrastrutture standard ben note e software sofisticati per gestire l'accesso alle risorse e ai dati, nonché la sicurezza degli stessi.

Un esempio di infrastruttura grid era [Seti@Home](#) (Search ExTraterrestrial Intelligence).

Grid Vs P2P

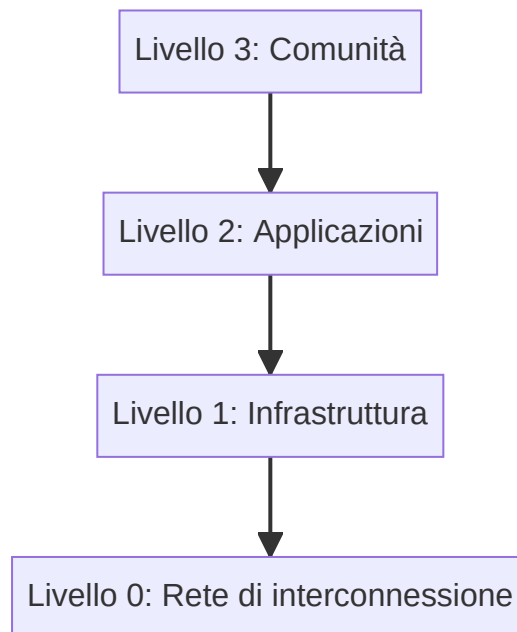
	Grid	P2P
Target	Piccole comunità di nodi per calcoli scientifici	Enormi comunità di nodi di utenti indipendenti ed incontrollabili
Risorse	Nodi molto potenti e omogenei	Nodi di potenza molto variabile e disomogenei
Servizi	Servizi sofisticati per accesso a risorse, sicurezza e accesso ai dati	Servizi discovery e condivisione di risorse
Infrastrutture	Infrastruttura standard	Nata dall'iniziativa di singoli per un fine preciso

Cloud

Un approccio moderno ormai divenuto una pratica standard a livello enterprise è quello del cloud computing. In questo schema, esiste un'organizzazione che fa da provider, mettendo a disposizione le proprie risorse agli utenti. Questo permette anche a piccole realtà di avere accesso, limitato in termini temporali, a macchine estremamente potenti e specializzate, senza doversi preoccupare della spesa che sarebbe invece necessaria per realizzare la stessa infrastruttura on premises. Ovviamente il prezzo da pagare, ma anche il vantaggio, è il non doversi preoccupare dell'hardware sottostante, delegando tutte le responsabilità al provider.

P2P

Livelli del P2P



La rete di interconnessione utilizza il sottostante sistema di interconnessione Internet e fornisce le funzionalità per la comunicazione fra peer.

A livello di infrastruttura, si realizzano tutti i protocolli e le direttive utilizzate nel routing a livello dell'overlay network.

Gli utenti interessati a partecipare alla rete dovranno scaricare delle applicazioni o client compatibili, per poter beneficiare dei servizi che la rete offre e poter contribuire a loro volta.

Non è da sottovalutare nemmeno l'aspetto sociale delle reti P2P. Più che in qualsiasi altro tipo di rete, il successo di un sistema distribuito dipende in larga parte dalla comunità che si viene a creare intorno ad essa.

Tipologie di sistemi P2P

- **Sistemi ibridi:** presenza di funzioni centralizzate (Napster, SETI@home)
- **Sistemi puri:** un grafo fortemente connesso caratteristico di un sistema completamente decentralizzato
- **Sistemi strutturati:** gerarchia a livelli che stabilisce il ruolo di ogni nodo. Introduce ruoli di coordinamento come supernodi o ultrapeer
- **Sistemi basati su Distributed Hash Tables (DHT):** utilizzano delle hashtable per i protocolli di instradamento dell'overlay network. In questo modo possono permettersi di essere totalmente decentralizzati
- **Sistemi gerarchici:** forte strutturazione

Valutazione di una topologia di rete overlay

- **Gestione:** costo del routing e della gestione e mantenimento della rete stessa
- **Coerenza delle informazioni:** le informazioni replicate devono essere coerenti fra di loro. Un possibile approccio è quello di rendere più probabile reperire informazioni più aggiornate
- **Fault tolerance:** la perdita di un nodo deve essere quanto più indolore possibile per la rete
- **Resilienza ad attacchi esterni:** quanto è difficile limitare la funzionalità del sistema
- **Sicurezza:** quali proprietà di sicurezza possono essere garantite dal sistema
- **Crescita:** quanto facilmente può crescere la rete con l'aggiunta di nuovi nodi
- **Scalabilità:** quanto grande può diventare la rete prima che le prestazioni inizino a degenerare

Tipi di topologia di rete P2P

Centralizzato

- La gestione è semplificata, in quanto controllata dal nodo server principale
- La coerenza è garantita se le informazioni sono contenute sul server principale. Diventa un po' più complicato se le informazioni sono sulle foglie
- Per far crescere il sistema è sufficiente aggiungere nodi foglia
- La fault tolerance è nulla, in quanto basta far cadere il server
- La sicurezza è facilmente gestibile
- La resilienza è debole se è possibile attaccare il nodo centrale
- La scalabilità è limitata dalle capacità del server centrale

Anello

- La gestione è semplice
- La coerenza può essere garantita eventualmente
- Per aggiungere un nuovo nodo ci dovrebbe essere un gestore centrale o un euristica
- Un nodo guasto può essere saltato, anche se potrebbero esserci dei limiti
- La sicurezza può essere gestita facilmente se introduciamo un gestore centrale
- Se abbiamo un gestore centrale è facile attaccarlo per interrompere il sistema
- La scalabilità influisce sulle prestazioni in maniera lineare, a meno di non utilizzare sistemi particolari di comunicazione come Chord

Gerarchica

- La gestione dipende da una catena di autorità
- La coerenza delle informazioni deve essere realizzata con l'ausilio di memorie cache
- La crescita è semplice ma bisogna che sia bilanciata, magari realizzando un RB-Tree

- La catena è tanto più debole quanto più in alto si va a colpire

Decentralizzata a grafo

- La gestione è molto difficile, così come la coerenza delle informazioni
- La crescita è molto semplice, chiunque può aggiungersi
- La fault tolerance è estremamente elevata
- La sicurezza è complessa e le ricerche potrebbero dover essere esaustive
- La scalabilità sembrerebbe essere infinita, ma in pratica nascono presto molti problemi

Approcci ibridi

Si tratta di uno dei primi approcci in grado di sfruttare quanto più possibile una rete P2P. Vi è ancora la presenza di un server centrale noto in grado di coordinare i peer fra di loro. In questa maniera non vi è la necessità di formare una overlay network fra i nodi, in quanto è sufficiente chiedere indicazioni al server. Solo dopo la comunicazione diventa diretta fra i due peer.

I vantaggi sono legati alla

- **semplicità:** la maggior parte del protocollo risiede nel server
- **efficienza:** non dovendo mantenere una overlay network, ai peer è richiesto un utilizzo di banda minimo durante il protocollo, se non quello necessario al trasferimento dei file
- **reperibilità:** se una risorsa è nota al server, può essere recuperata

D'altra parte, gli svantaggi sono:

- **single point of failure:** se il server cade, l'intera rete smette di funzionare
- **scalabilità:** la crescita della rete è limitata dalle capacità computazionali del server
- **manca di anonimato:** il server conosce il comportamento di tutti i peer, in quanto devono passare da lui. Si richiede una fiducia implicita nel comportamento corretto del server

Anello con nodi esterni

Mantiene un anello di dimensioni contenute al centro, a cui aggiungere dei nodi esterni che si appoggiano al nucleo centrale

Centralizzata con decentralizzata

Nodi più grossi che sono a capo di diversi nodi periferici, in grado di comunicare con altri server in una rete decentralizzata di dimensioni adeguate.

Sistemi non strutturati

Sono basati su peer indipendenti che cooperano tra loro. Il coordinamento e il mantenimento della overlay network devono essere completamente decentralizzati. Ci vogliono comunque dei nodi conosciuti (e quindi esposti) che facciano da punto d'ingresso nella rete per nuovi nodi.

Una query in questo tipo di sistema necessita l'utilizzare del flooding. Per impedire che il flooding esca fuori controllo e rimbalzi all'infinito, al pacchetto viene aggiunto un TTL e un id, in maniera che i nodi possano tranquillamente scartare pacchetti troppo vecchi o già visti.

I vantaggi di questo tipo di architettura sono:

- **ottima resilienza:** anche se un sottoinsieme di nodi sparisce o smette di funzionare, il sistema continua ad operare
- **dinamicità:** molto facile entrare ed uscire dalla rete
- **impossibile da bloccare**

D'altra parte, gli svantaggi sono:

- **mantenimento complesso:** mantenere un overlay network senza un coordinamento centralizzato può essere particolarmente complesso
- **scalabilità delle query:** a causa del flooding, la banda del canale può saturarsi molto velocemente
- **poca protezione da agenti malevoli:** può essere difficile individuare e punire nodi che si limitino a sfruttare le risorse della rete o addirittura agiscano in maniera nociva
- **anonimato non garantito**

Sistemi strutturati

Sono stati introdotti per risolvere i problemi dei sistemi non strutturati. La differenza principale sta nell'introduzione di una differenziazione fra i peer, che si distinguono in nodi di primo livello (pochi e potenti) e nodi di secondo livello (molti e meno potenti).

I nodi di primo livello sono detti **super-node**, mentre quelli di secondo livello **leaf-node**.

I leaf-node non formano alcun tipo di rete ma sono connessi ad uno o più super-node di riferimento.

I super-node agiscono all'interno di una overlay network di super-node e agli occhi dei leaf node si comportano come un server centrale in grado di indicizzare le risorse e rispondere alle query.

Distributed Hash Tables (DHT)

I protocolli che scelgono questo approccio utilizzano una funzione di hashing per realizzare uno spazio di indirizzamento flat di dimensioni enormi. La risorsa corrispondente ad una chiave può essere un file o persino un altro nodo qualsiasi della rete, o precisamente il peer che la possiede.

Memorizzare l'intera tabella richiederebbe una quantità di memoria immane. Per questo i protocolli di routing sono studiati per funzionare considerando solo la conoscenza parziale che ogni nodo ha della tabella.

Ad ogni peer è quindi assegnata una piccola parte dello spazio di indirizzamento, che spesso si traduce anche nella responsabilità di gestire le risorse che dovrebbero essere locate lì. Un fattore di replicazione stabilito a priori assicura che ci siano più nodi in grado di soddisfare una richiesta qualora il peer originale non fosse disponibile.

Le primitive minime necessarie per realizzare questo tipo di rete sono i comandi `get(key)` e `put(key, value)`. Come si può intuire, il primo comando ottiene la risorsa che nella Hash table è associata alla chiave specificata. Nel secondo caso viene inserita una nuova risorsa con la chiave indicata.

Protocolli P2P

Napster

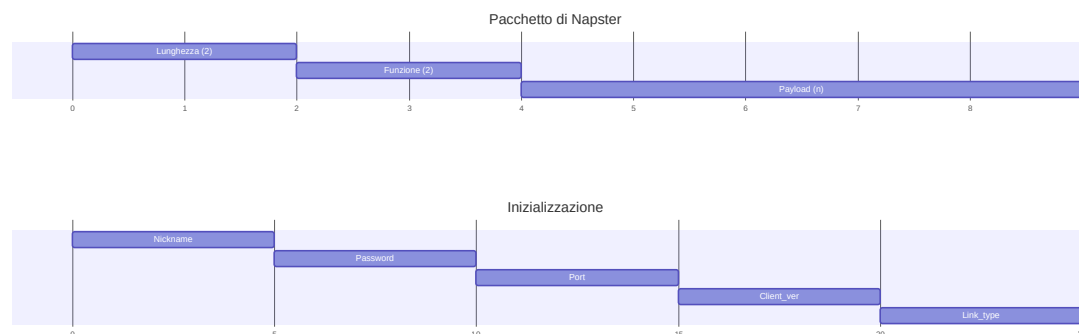
Architettura ibrida

Napster fu il primo programma per reti P2P a raggiungere una grande diffusione. Il suo creatore, S. Fanning, lo voleva usare per condividere e scambiare file .mp3. Utilizza dei server centrali per coordinare i peer. I peer comunicano al server la loro presenza e ottengono così la lista di altri peer che possiedono le informazioni cercate. Sarà poi compito dei peer mettersi in comunicazione l'uno con l'altro per lo scambio effettivo di dati. Non vi è alcuna rete overlay realmente presente tra i peer.

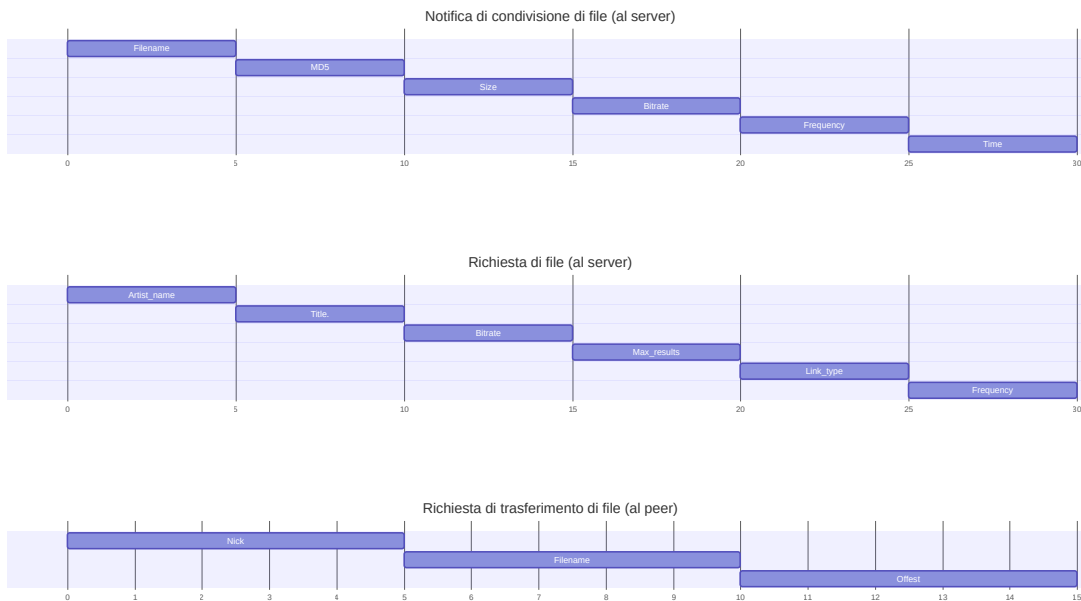
I passi del protocollo erano:

1. Connessione al server Napster
2. Upload della propria lista di file condivisibili sul server
3. Ricerca tramite keyword sul server
4. Invio di ping al fine di trovare il peer migliore disponibile con i dati d'interesse
5. Download dei file tramite comunicazione diretta con il peer selezionato

Pacchetti

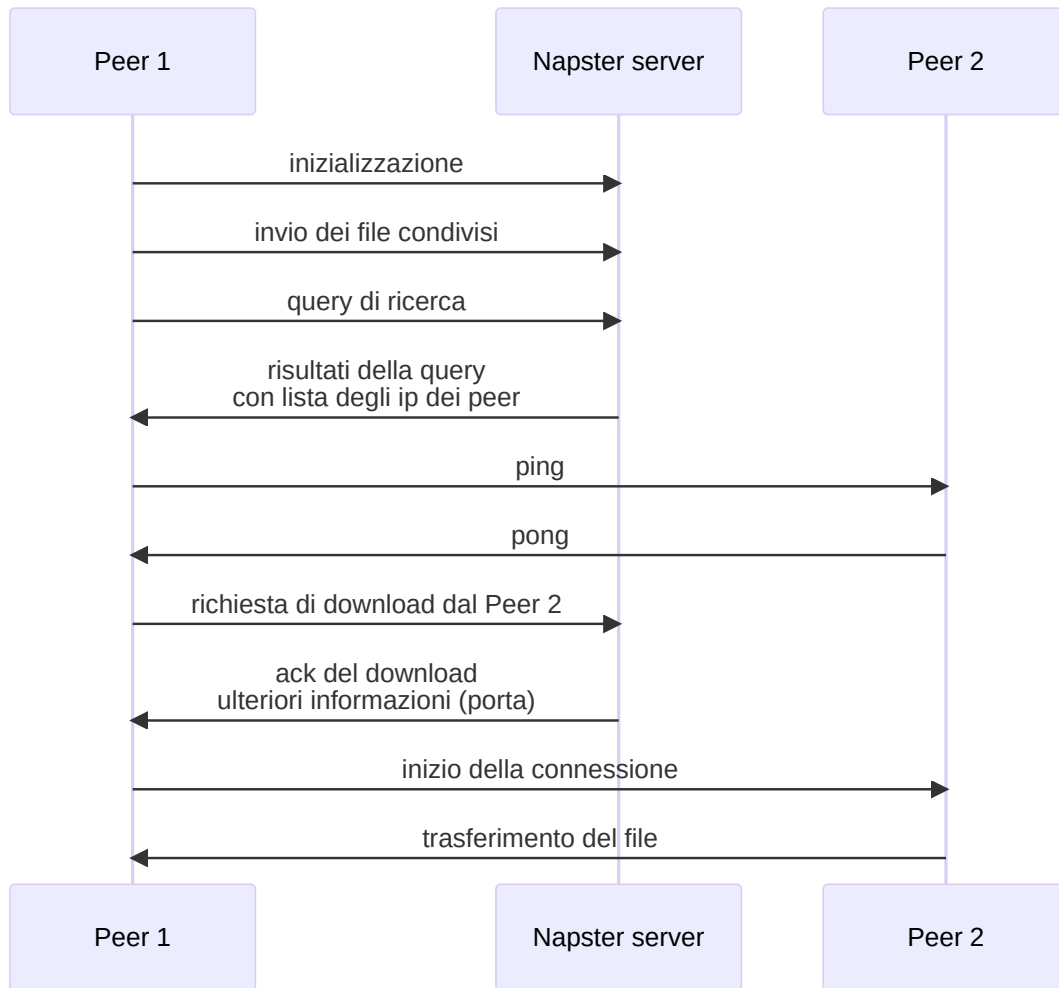


Il *Link_type* indica il tipo di connessione che l'utente dichiara di avere, e quindi la sua affidabilità e velocità. Più è alto il valore, compreso comunque fra 0 e 10, migliore è la connessione che si afferma di possedere.



La richiesta di trasferimento permette di stabilire un offset da cui fare partire il download, per poter riprendere da dove si era lasciato nel caso di un'interruzione imprevista della rete o di un errore.

Protocollo



Architettura

Questo approccio ibrido con un server centrale rappresenta uno dei modelli più semplici per realizzare una rete P2P. Il server si fa infatti carico della gestione e del coordinamento della rete, rimuovendo la necessità di costruire e gestire una overlay network fra i peer.

La presenza di un server centrale rappresenta però un **single point of failure**, in quanto è sufficiente attaccare il server per impedire il corretto funzionamento di tutto il protocollo.

Napster era stata pensata specificatamente per file .mp3. Estenderla anche a file generici avrebbe introdotto ulteriore complessità.

Il protocollo non prevede alcun tipo di sicurezza o anonimato, prestando il fianco a tracciamenti e comportamenti malevoli dei partecipanti.

Gnutella 0.4

Sistema non strutturato

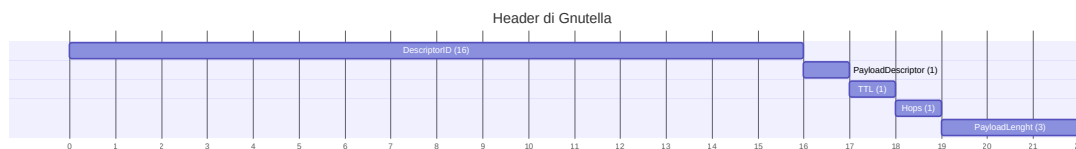
I peer, chiamati **servents**, sono connessi tra loro in una overlay network. Per il collegamento iniziale è necessario conoscere almeno un nodo che già fa parte della rete.

Sebbene sia rimasta closed source, con il tempo sono nati numerosi client frutto del reverse engineering su quello ufficiale.

L'architettura di Gnutella è veramente distribuita, per cui non esiste un nodo centrale in grado di coordinare la rete. Ogni peer ha la necessità di mettersi in contatto con altri nodi della rete, spesso quattro, a cui inoltrerà le proprie ricerche, che percorreranno poi la rete grazie ad un meccanismo di flooding.

Pacchetti

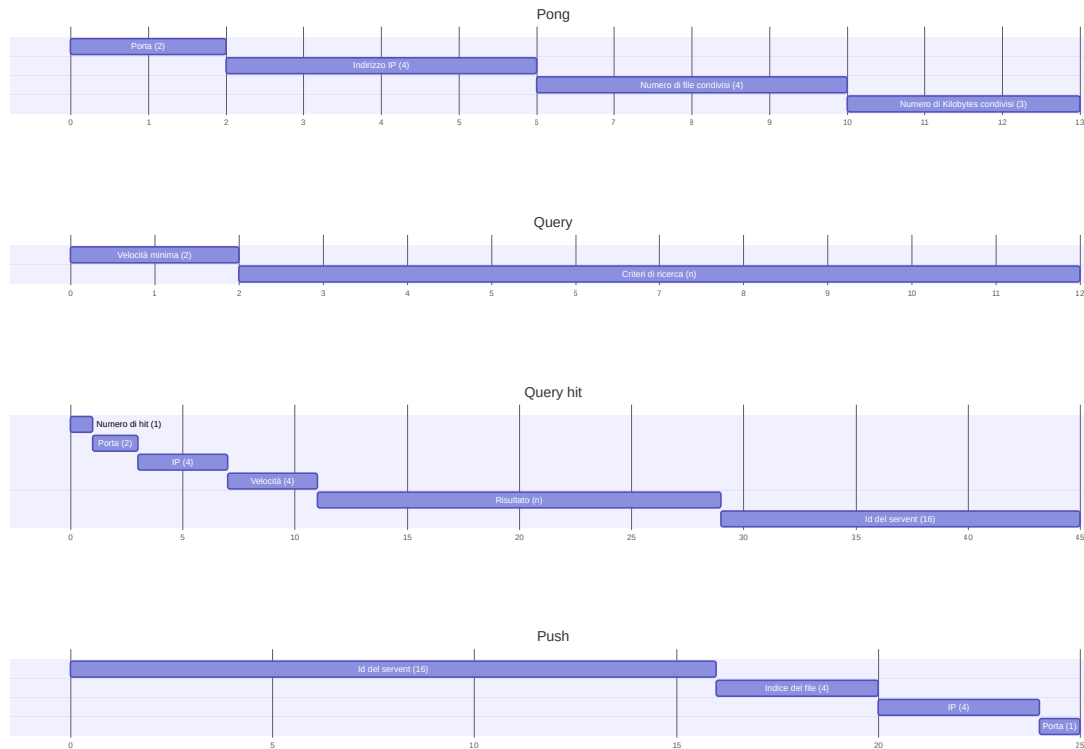
I pacchetti previsti dal protocollo di Gnutella sono composti da un header uguale per tutti e da un payload che variano a seconda della tipologia di messaggio che si vuole inviare.



I possibili tipi di messaggio sono

- **ping**: usato per scoprire i nodi sulla rete tramite un meccanismo broadcast. Contiene solo l'header
- **pong**: risponde ad un ping con il percorso a ritroso. Viene anche espressa la qualità autocertificata del peer che risponde, stabilita a partire dal numero di file condivisi e dalla loro dimensione
- **query**: ricerca di un file nella rete
- **queryhits**: risposta ad una query con le informazioni cercate. Dopo aver ottenuto i risultati, il download effettivo avviene tramite HTTP.
- **push**: richiesta di apertura di una porta per bypassare il NAT

Mentre il pacchetto di ping contiene solo l'header, gli altri pacchetti presenteranno un payload specializzato



Regole

Per il buon funzionamento del protocollo per il mantenimento della overlay network, i peer devono seguire delle regole ben precise:

1. Tutti i peer memorizzano un descrittore di 128 bit per ogni messaggio, per evitare di replicare inutilmente i messaggi già visti.
2. Tutti i messaggi di Pong, QueryHit e Push devono essere inviati solo in risposta ad un Ping, Query e QueryHit rispettivamente, e solo seguendo il percorso del messaggio precedente.
3. Ogni ping e query viene inoltrato a tutti i peer con cui si ha un contatto diretto eccetto che al da cui lo si ha appena ricevuto
4. Ad ogni passaggio il TTL viene decrementato, mentre il valore di Hops incrementato. Se il TTL è 0, il messaggio viene scartato
5. Se un servient riceve un messaggio con un id da 128 bit già visto, il messaggio viene scartato

Una volta ricevuto il QueryHit dal peer che possiede la risorsa, il trasferimento di quest'ultima viene effettuato tramite HTTP con una connessione diretta.

Fenomeno del Free riding

Il tallone d'Achille di Gnutella è la mancanza di incentivi verso un comportamento corretto, nonché di punizioni per chi viola il protocollo. Al contrario, si è manifestato presto il problema del free riding, cioè di utenti che pretendevano di ricevere i file dagli altri peer senza condividere nulla o condividendo spazzatura per aumentare il proprio valore agli occhi degli altri peer. I pochi utenti effettivamente attivi hanno presto abbandonato il sistema, lasciandolo a morire.

Flooding e colli di bottiglia

Un altro problema a cui Gnutella era soggetta è quello del flooding massivo. Se le dimensioni della rete fossero cresciute a sufficienza, sarebbe stato impossibile gestire tutti i messaggi, specialmente quelli di query, a causa del numero abnorme di messaggi di risposta che avrebbero generato.

Il problema nasce dall'uso smodato del flooding come unico mezzo per permettere al peer di entrare in contatto con altri peer non immediatamente adiacenti. Aumentare il TTL significa aumentare esponenzialmente il numero di messaggi che viaggia nella rete, saturandola. D'altro canto, diminuirlo significa ridurre la "visione" del peer, impedendogli di mettersi in contatto con nodi più lontani.

Il TTL può anche essere sfruttato per deanonimizzare la comunicazione: vedere un messaggio che ha ancora il TTL integro vuol dire conoscere il mittente originale.

Gnutella 0.6

Sistema strutturato

Architettura

Per migliorare Gnutella ci si è proposto di ridurre il numero di messaggi scambiati,

Vengono quindi definiti due tipi di nodi: **ultrapeer** e nodi **foglia**.

Per essere considerato un ultrapeer, è necessario avere una buona connessione, non essere dietro firewall e avere un tempo di permanenza nella rete elevato. Un nodo che rispetta i requisiti elencati, si autocandida a diventare un nodo ultrapeer. Dopo averlo comunicato al proprio ultrapeer, se le condizioni sono propizie (il numero di nodi nella regione è sufficiente a giustificare la promozione) è la rete stessa ad accettare la sua promozione.

Gli ultrapeer utilizzano un protocollo di indirizzamento estremamente simile a Gnutella 0.4.

Un singolo ultrapeer può arrivare a gestire anche una trentina di nodi foglia e può essere connesso in maniera diretta con un massimo di 32 ultrapeer.

I nodi **foglia** sono connessi solo agli ultrapeer, ma viene fornita loro una lista di ultrapeer a cui collegarsi nel caso si perda la connessione con l'ultrapeer corrente.

I nodi foglia sono connessi con uno o più ultrapeer, ma non sono connessi tra di loro.

Ricerca di un file

Quando un nodo foglia richiede una risorsa, comunica la query al proprio ultrapeer. Questo farà il flooding della richiesta in maniera simile a ciò che avviene in Gnutella 0.4.

Filtro di Bloom

*Si tratta di una struttura dati probabilistica efficiente in termini di spazio. Viene utilizzata per verificare velocemente se un elemento **potrebbe** essere contenuto in un set.*

Utilizzando questo sistema, è possibile ottenere dei falsi positivi (l'elemento non si trova dove pensavo) ma non dei falsi negativi (se si ottiene l'informazione che l'elemento non ci sia, questa affermazione è certa).

Il protocollo di query prende il nome di Query Routing Protocol (QRP). Ogni nodo prepara una Query Routing Table (QRT). Si tratta di una hash table composta da fino a 2 milioni di slot. Gli ultrapeer ricevono le QRT da tutti i nodi foglie e le fondono insieme. La tabella risultante viene scambiata con tutti gli ultrapeer vicini.

L'ultrapeer utilizza un **filtro di Bloom** che aveva ricevuto dai nodi foglia per verificare se la query contiene delle keyword che ricadono in celle piene del filtro di Bloom. Se c'è un match, la ricerca sarà reindirizzata al nodo periferico che ha dato al server il filtro di Bloom contenente la keyword.

Dato che la richiesta include anche l'indirizzo ip del nodo richiedente, la comunicazione per l'upload/download può avvenire direttamente tra i nodi foglia, senza passare più dagli ultrapeer.

In questa maniera il traffico di flooding legato alla query è ridotto al minimo e relegato ai soli ultrapeer, mentre le foglie non sono coinvolte.

È stato aggiunto anche un descrittore per invalidare e quindi forzare l'aggiornamento della routing table degli ultrapeer.

Gnutella 2

Sistema strutturato

Sviluppato come evoluzione di Gnutella 0.6, il protocollo diventò incompatibile con la versione precedente a causa del cambiamento nel formato dei messaggi.

Fra le principali differenze con Gnutella 0.6 vi è un differente meccanismo di ricerca e l'adozione di **UDP** come protocollo di trasporto, utilizzato per alcune comunicazioni che potevano supportare l'inaffidabilità del protocollo. Eventualmente poteva essere reso più affidabile tramite l'invio di un ACK di risposta.

Architettura

Un **hub**, l'equivalente di un ultrapeer, gestisce anche 300-500 **foglie**, ed è connesso ad altri 5-30 hub, mentre le foglie sono connesse ad almeno 2 hub. Generalmente la ricerca si limita agli hub vicini, senza andare a fare flooding sull'intera rete.

Ogni hub mantiene una query table con delle informazioni aggiornate provenienti sia sulle foglie che gestisce, sia dagli hub vicini.

Operazione di query

Per effettuare una ricerca, un nodo foglia invia la query al suo hub di riferimento. Questo consulta la query table per verificare se conosce un nodo, hub o foglia, che potrebbe essere in possesso della risorsa.

Se lo trova, trasferisce la query al nodo in questione se il nuovo target è un nodo hub, o permette ai due nodi foglia di entrare in comunicazione diretta, così che possano condividere la risorsa.

Se non lo trova, piuttosto che continuare la ricerca con un flooding, l'hub risponderà alla foglia con una lista di altri hub da consultare per provare a proseguire nella ricerca. In questa maniera sta alla volontà della foglia continuare la query. Si ottiene una riduzione dello spreco indotto dal flooding al costo di un maggiore coinvolgimento dei nodi foglia che comporta, fra le altre cose, una perdita praticamente totale dell'anonimato.

Per evitare la cattiva condotta da parte di una foglia, questa, per poter fare una richiesta, deve procurarsi una **query key** dall'ultrapeer di riferimento, che deve autorizzare la richiesta. Le chiavi possono essere accumulate per poter essere utilizzate in un momento di picco, sebbene i singoli token abbiano comunque una scadenza temporale.

FastTrack

Sistema strutturato

Nasce dal voler superare i limiti di Napster e altri competitor.

Si tratta di una rete strutturata composta da due gruppi di nodi: il gruppo dei **supernodi** e quello dei **nodi ordinari**, in una struttura molto simile a quella di Gnutella 2.

Del protocollo si sa poco a causa della volontà degli autori di mantenere tutto quanto più privato possibile, anche attraverso la cifratura.

Ogni nodo ordinario comunica con il supernodo comunicando i dati relativi ai file che condivide, inclusi nome, dimensioni, MD5 e una descrizione.

Il download del file avviene tramite frammenti, per cui è possibile parallelizzare richiedendo frammenti diversi a più nodi.

OpenFT

Versione open source di FastTrack, sebbene incompatibile con quest'ultima. Introduce la presenza di tre tipi di nodi:

- **user node:** nodi client degli utenti
- **search node:** mantengono gli indici dei file che sono condivisi dagli user node ad essi connessi
- **index node:** sono in numero molto limitato e hanno lo scopo di indicizzare i search node

EDonkey

Si tratta di un approccio gerarchico che vede la distinzione dei nodi in server e client. I primi formano una rete non strutturata con archivi separati e si occupano di gestire la ricerca delle risorse.

Un client può richiedere una risorsa ai server a cui è connesso. Se la ricerca non si conclude con successo, può passare a quelli successivi.

EMule

Si tratta del software client più diffuso per EDonkey. Prese presto vita propria ed estese il protocollo appoggiandosi ad una rete Kademlia.

Come per EDonkey, viene utilizzata un'architettura fortemente improntata al modello client-server, ed utilizza sia TCP che UDP.

Identificativi

Emule utilizza una serie di identificativi, applicati sia ai client che ai file:

- **ClientID:** un identificatore a 32 bit, fornito dal server nel momento della prima connessione. Rimane valido per tutto il tempo di vita della sessione
 - **LowID:** valore generato dal server di connessione
 - **HighID:** calcolato sulla base del proprio indirizzo ip:

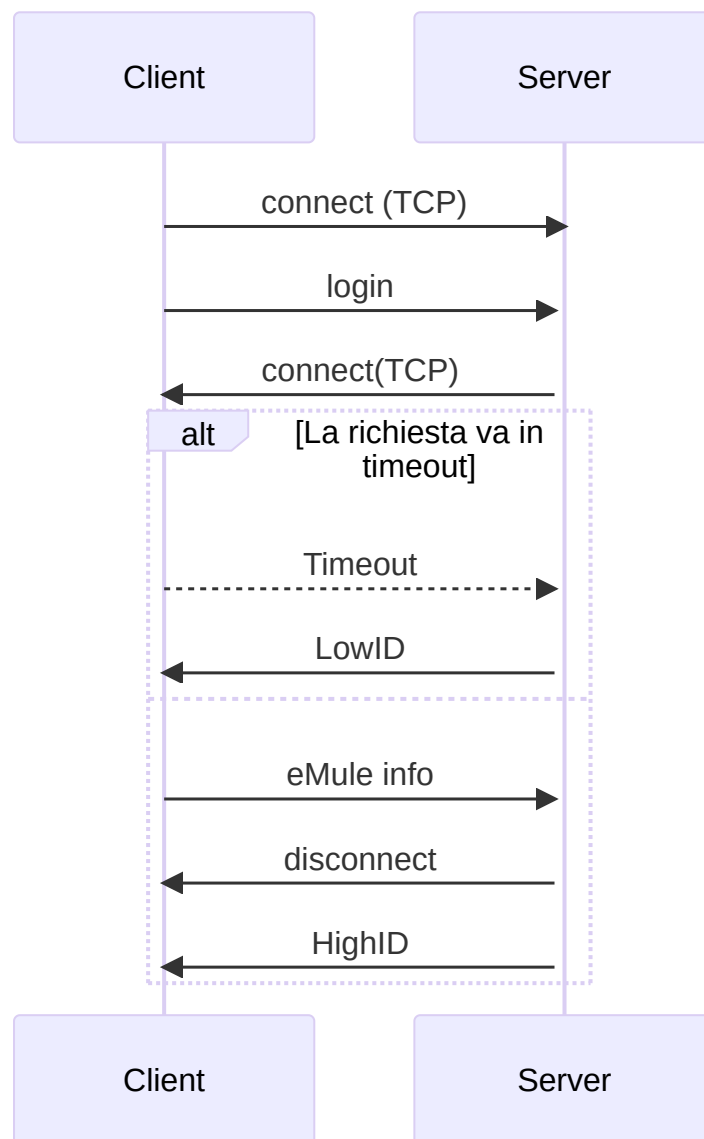
$$IP = x.y.z.w \rightarrow x + 2^8 \cdot y + 2^{16} \cdot z + 2^{24} \cdot w$$

- **UserID**: valore a 128 bit prodotto dalla concatenazione di numeri random (tranne il 6° e 15° byte). Può identificare un client in più di una sessione. È utilizzato per il sistema di crediti
- **FileID**: hash di un file che lo identifica in maniera univoca sulla base del contenuto. Utilizza MD4, suddividendo il file in chunk da 9.28 MB

Startup

La prima volta che un nodo si introduce nella rete, si connette tramite TCP ad un server, da cui riceve una lista di altri server attualmente attivi.

Per determinare il **ClientID** che il client riceverà, il server effettua un tentativo di connessione al client durante il login del client.



Il meccanismo callback consente ad un client con **LowID** di essere contattato mantenendo un canale TCP aperto fra client e server. Nel momento in cui viene richiesta la risorsa, il server notificherà il nodo **LowID** con un messaggio definito di **callback**. Quest'ultimo si metterà in contatto direttamente con il nodo richiedente.

Utilizzando tale principio, è possibile mettere in comunicazione tra di loro anche due nodi che abbiano entrambi un **LowId**. Quest'ultima opzione, sebbene funzionante, corre il rischio di sovraccaricare il server, per cui è stata presto abbandonata.

Ad intervalli di tempo regolari, i client verificano lo stato dei server che conoscono per assicurarsi che siano ancora online. Il ping viene effettuato tramite UDP, e contiene una nonce che evita duplicati o errori nella trasmissione.

Questo traffico di mantenimento è reso il più leggero possibile. L'obiettivo è quello di non superare il 5% del traffico totale della rete.

Gestione dei file

I file sono identificati dal loro hash, prodotto tramite MD4. I file vengono suddivisi in parti da 9MB e, ulteriormente, in chunk da 180KB. Ad ogni chunk è associato un hash parziale. L'hash identificativo del file è prodotto a partire dagli hash parziali dei singoli chunk, in una maniera molto affine ai Merkle Tree.

Nel caso ci fossero problemi con il file ottenuto in download, verificabile tramite il confronto fra hash dei dati ricevuti e id dichiarato del file, è previsto un sistema di **Intelligent Header Correction** (IHC). In questa maniera è possibile interrompere il download e ricaricare i singoli chunk a partire dal primo.

La procedura di correzione è stata ulteriormente affinata in **Advanced Intelligent Header Correction** (AIHC). In questa versione, nel caso si verifichi un problema nel download della risorsa, è possibile utilizzare gli hash parziali per identificare i chunk danneggiati e ricaricare solo le parti necessarie.

Query

La ricerca avviene per lo più in maniera locale, richiedendo ai server di riferimento. In alternativa, il client può provare ad allargare la ricerca globalmente, anche se l'operazione sarà ovviamente più lenta.

Una volta trovato l'oggetto cercato, il client può minimizzare il tempo di download richiedendo in parallelo diversi chunk da nodi differenti.

Esiste un sistema di rarità per i chunk. Questi possono essere **diffusi**, **rari**, **estremamente rari**. Ovviamente i chunk più rari vengono privilegiati nell'ordine di download.

Reputazione per la coda di upload

La coda di upload si comporta come una coda di priorità. Per ogni client che richiede una risorsa viene calcolato un punteggio come $\text{score} = \text{rating} \times \text{seconds in queue} / 100$. Tutti gli utenti partono con un rating iniziale pari a 100. Gli utenti bannati possiedono un rating pari a 0.

Al rating è applicato un modificatore definito come
$$\begin{cases} \text{uploaded} \times 2 / \text{downloaded} \\ \text{uploaded} + 2 \end{cases}.$$

Si sceglie la formula che produce il risultato minore.

Il modificatore può venire alterato dai crediti che il nodo si guadagna nei confronti di un peer. Ad esempio un nodo in download riceve un bonus di 200 per 15 minuti per evitare di venire scalzato da nodi nuovi prima di aver completato l'operazione, a patto che il richiedente invii un ping UDP ad un intervallo prestabilito. Ricevere un chunk da un altro nodo fa aumentare i suoi crediti, che possono essere un valore compreso fra 1 e 10. Anche la qualità del chunk condiviso contribuisce a far guadagnare crediti in proporzione alla rarità.

Il sistema dei crediti è utilizzato solo nella comunicazione diretta fra due nodi. In questa maniera è sufficiente che il nodo che assegna i crediti memorizzi localmente lo storico dell'altro nodo, evitando il rischio di farsi ingannare da crediti fasulli. Si disincentiva anche il meccanismo del free riding e si incentiva la collaborazione attiva.

BitTorrent

Sistema strutturato

Il protocollo fu presentato ad una conferenza del 2002 da Bram Cohen scritto in python.

L'idea era quella di capitalizzare sul fatto che la popolarità di un file è generalmente molto limitata in termini spaziali e temporali.

Ci si concentra in particolare sulla condivisione di file di grandi dimensioni. Questo tipo di file, infatti, causava spesso problemi per i protocolli precedenti. Si è anche presa la decisione architetturale di gestire in maniera separata le operazioni di search and download.

Architettura

I nodi che partecipano al protocollo si suddividono in base al ruolo che hanno nell'operazione di ricerca:

- **tracker**: nodo speciale che conosce i peer che intendono partecipare
- **seeder**: nodo che possiede tutto il file
- **peer**: nodo che partecipa all'upload o al download di parti del file

Lo **swarm** è una lista di nodi interessati ad una stessa risorsa che il **tracker** comunicherà al peer che l'ha contattato. Non tutti i nodi possibili fanno parte dello swarm, ma ogni peer riceverà dal tracker solo un subset. Di conseguenza ogni nodo si interfacerà con uno swarm diverso da quello di un altro peer, anche se ci saranno dei nodi in comune.

La salute dello swarm è calcolata come il numero di copie complete della risorsa presenti nella rete.

Quando uno **swarm** di peer richiede una risorsa al seeder, questo suddivide il file in blocchi da 256 kB o multipli e sotto blocchi da 16 kb. Le dimensioni possono essere adattate alle necessità, specificandole nel file .torrent. Ad ogni nodo richiedente viene affidato solo un blocco del file.

Ogni nodo può contattare i peer del suo swarm per scambiare gli indici dei blocchi posseduti. Così viene a conoscenza della distribuzione dei pezzi dei blocchi e la loro rarità. Comunicando con gli altri peer che fanno parte dello swarm, tutti i nodi sono in grado di ottenere l'intero file, distribuendo lo sforzo in maniera piuttosto omogenea.

Operazioni

Ricerca

- gli utenti trovano il file **.torrent** con metodologie esterne al protocollo, nella searching phase. Questo file contiene l'indirizzo del **tracker** e altre dati relativi al file.

- il peer contatta il tracker.
- il tracker fornisce una lista random di altri peer, chiamata **swarm**.
- il peer contatta tutti i peer che conosce e inizia le operazioni scambio di dati.
- periodicamente ogni peer ricontatta il tracker per avere un nuovo swarm.

Pubblicazione

- **pubblicazione**: viene creato un file .torrent che definisce un tracker server.
- **search**: viene effettuata una richiesta al di fuori dal protocollo per procurarsi il file .torrent, spesso attraverso un forum o sito web.
- **join**: si contatta il tracker per ottenere una lista degli altri peer.
- **fetch**: si contattano i singoli peer dello swarm per scambiare le parti di file reciprocamente possedute.

Swarm

- si fa parte dello swarm
- si richiedono blocchi a caso, possibilmente non rari
- si utilizza l'algoritmo **Rarest Piece First** (RPF) per dare la priorità ai peer in grado di offrire dei blocchi più rari
- si attiva una **Endgame Mode**, inviando in broadcast una richiesta che specifica i pochi blocchi mancanti

Tit-for-Tat

La condivisione delle risorse su BitTorrent utilizza la strategia **tit-for-tat**. I nodi si comportano in maniera più disponibile, condividendo le proprie risorse, verso i peer da cui ricevono i blocchi cercati.

Per evitare il potenziale problema del free riding, i nodi implementano un **algoritmo di choking** che procede come segue:

- una connessione viene soffocata quando ci si rifiuta di fornire dati, sebbene la connessione rimanga effettivamente attiva
- ogni peer cerca di massimizzare la propria velocità di download, scartando le connessioni in download più lente

- ogni 10 secondi i peer controllano le proprie connessioni (tipicamente 4) valutando il rapporto download/upload. I peer con valori più alti vengono unchoked
- si scartano man mano i peer più scarsi
- **be optimistic**: ogni 30 secondi si consente alle connessioni più lente di scaricare. Serve anche a far partire nuovi arrivati e testare nuove connessioni. Può essere applicato nella **Endgame Mode** per aumentare le possibilità di entrare nelle grazie di un peer che abbia i pochi blocchi mancanti

Un peer che prova a fare free riding sarà in grado di proseguire nel suo comportamento con difficoltà sempre crescente, subendo una riduzione progressiva della velocità del download.

Un effetto collaterale di questo tipo di reputazione è che vengono a formarsi gruppi di peer che hanno connessioni alla stessa velocità. Un peer con una connessione lenta non rimarrà a lungo nello swarm di peer più veloci.

Freenet

Sistema non strutturato

Freenet è una rete che nasce con l'obiettivo principale di creare una rete anonima. Si tratta di un sistema per scrivere e leggere file da internet senza che si possa risalire a chi li ha scritti, chi li conserva sul disco e chi li recupera.

La rete è in grado di garantire anonimato per produttore e del consumatore, resilienza a tentativi di attacco da terze parti, efficienza in termini di storage e routing, funzioni di rete decentralizzate. Le performance sono limitate rispetto a quelle di altri protocolli P2P.

Architettura

La rete Freenet è completamente distribuita. Ogni partecipante mette a disposizione il proprio spazio di archiviazione. L'intera rete può essere quindi vista come un grande file system cooperativo e distribuito.

Ogni richiesta viene presa in carico dal nodo che la riceve, che si occupa di stabilire il prossimo salto. Le tabelle di routing si adattano ai cambiamenti della rete, aggiornandosi ad ogni richiesta.

Storage

Mentre protocolli come Gnutella e Napster prevedono che il file sia memorizzato per intero nei peer che lo condividono con gli altri, in Freenet ogni nodo mette a disposizione il proprio spazio per memorizzare solo parte della risorsa. Quando un nuovo file viene inserito nella rete, viene suddiviso in blocchi e distribuito fra i vari nodi partecipanti. Anche se il nodo che ha originariamente aggiunto il file andasse offline, sarebbe ancora possibile recuperare la risorsa. Inoltre, ad ogni file che si memorizza nella rete, viene associato un **Globally Unique Identifier** (GUID) calcolato mediante SHA-1, che permette la sua ricerca.

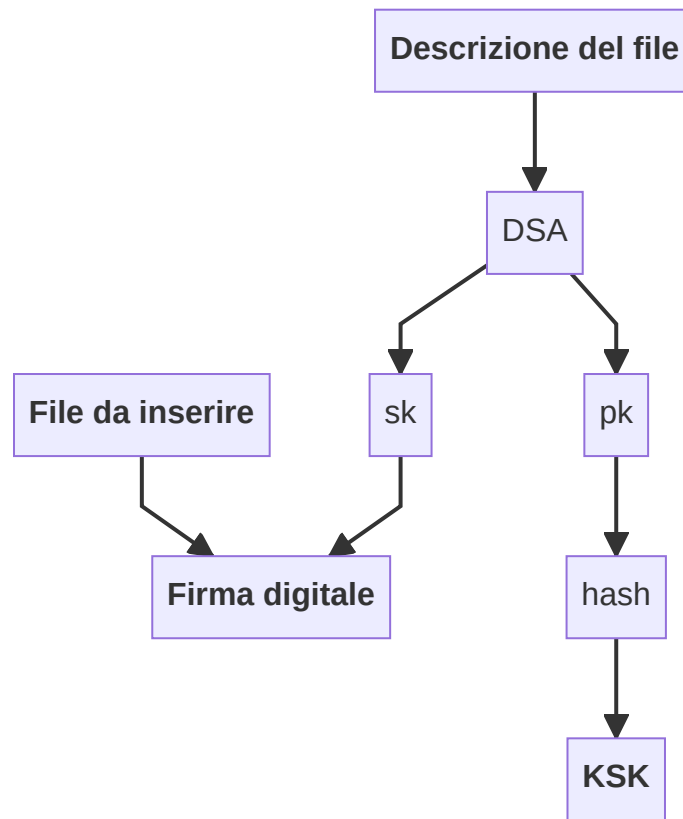
Identificativi

Freenet utilizza tre tipi di identificativi GUID che permettono di scegliere in che modalità il file sarà indirizzato.

Keyword Signed Key (KSK)

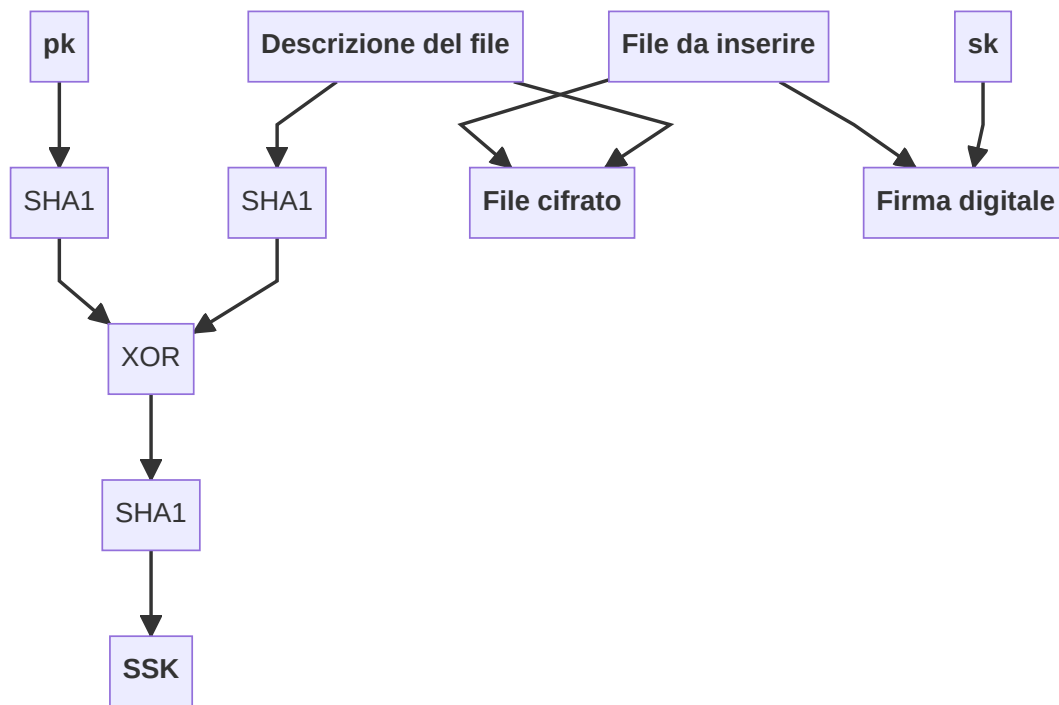
Si tratta di una stringa descrittiva, scelta dall'utente, che descrive brevemente il file. Da questa si ottiene una coppia di chiavi pubblica/privata applicando una Key Derivation function (DSA). La chiave privata viene utilizzata per firmare il file e quindi garantire la sua integrità ed autenticità, mentre dalla chiave pubblica, tramite hashing, si ottiene il GUID.

La scelta arbitraria della stringa descrittiva potrebbe causare una collisione, e conseguente sovrascrittura dei file. Dato che chiunque potrebbe generare la stessa coppia di chiavi, la risorsa che utilizza questo schema non ha un effettivo proprietario.



Signed Subspace Key (SSK)

Permette la creazione di un namespace personale. L'utente crea una coppia di chiavi ed una stringa descrittiva. La stringa viene usata per cifrare il file, mentre la chiave privata firma il file, impedendo a chiunque non la possiede di sovrascrivere il file. Volendo pubblicare dei dati in sola lettura, è sufficiente condividere la stringa descrittiva e la chiave pubblica, permettendo agli utenti di generare liberamente la SSK, ma non la firma digitale.

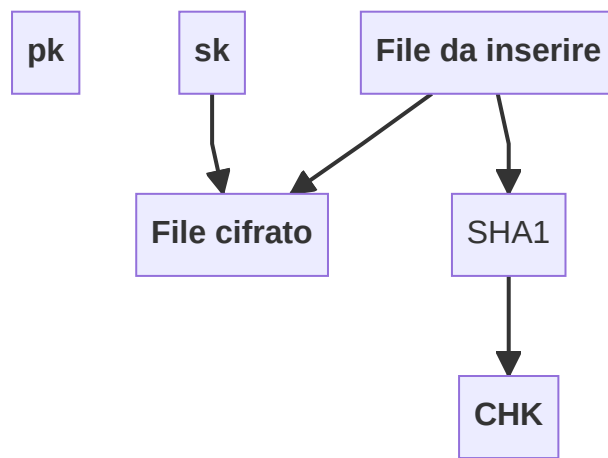


Content Hash Key (CHK)

Utilizzare questo tipo di chiave permette l'aggiornamento e lo splitting del file. La chiave CHK viene generata dall'hash del file, consentendo di dare ad ogni file una chiave praticamente univoca.

Il file viene cifrato utilizzando la crittografia asimmetrica. Vengono pubblicati solo la CHK e la chiave pubblica.

È possibile combinare più approcci. Ad esempio è comune creare un file che contiene una lista di CHK che viene pubblicato con SSK. In questa maniera, quando un file viene aggiornato (o un file molto grande viene diviso in più sezioni) è possibile aggiornare il file SSK. Senza dover cambiare alcunché, gli utenti troverebbero nello stesso file SSK i riferimenti alla nuova versione (o a tutte le parti) del file CHK.



Routing

Il routing è pilotato dal valore della chiave associata al file da cercare. Si tratta di un routing adattivo, in cui le tabelle vengono aggiornate durante la ricerca delle chiavi, per cui il sistema va verso configurazioni più efficienti. Ad ogni messaggio si associa un HTL (Hop To Live) ed una nonce che i nodi memorizzano per evitare di inoltrare più volte lo stesso messaggio.

Inserimento di un file

Per inserire un file nella rete, l'utente assegna una chiave K al file. Se la chiave è già presente sul nodo locale, è a conoscenza del fatto che avverrà un conflitto. Se, al contrario, la chiave non esiste, allora invia al nodo che possiede la chiave più vicina ed invia un comando `INSERT( $K$ , HTL)`. Il messaggio viene inoltrato ulteriormente fino a quando il valore di HTL si azzerava. L'ultimo nodo della catena accetta di memorizzare il dato, e si prova a ripercorrere la catena notificando dell'avvenuto inserimento. I nodi notificati memorizzano la nuova chiave, sapendo che il nodo che gli ha passato il messaggio è in possesso della risorsa.

Si noti che, per evitare che il HTL sia troppo rivelatore, è prevista una possibilità statistica che il suo valore non sia diminuito, rendendo molto più difficile avere la certezza del percorso che ha fatto il pacchetto.

C'è anche la possibilità che un nodo intermedio, dopo aver ricevuto una ricerca che ha successo, memorizzi il risultato di questa. Dato che la scelta è statistica, la probabilità che il file venga duplicato dipende dalla popolarità dello stesso, di fatto migliorando l'efficienza della rete.

Ricerca di un file

Quando un nodo riceve una query per il file con chiave k

- se il nodo possiede il file, lo restituisce, aggiungendo anche il proprio identificativo
- se il nodo non possiede il file, esso esamina la propria routing table per inoltrare la richiesta al nodo che dovrebbe possedere un file con la chiave più vicina a quella richiesta. Nel caso riceva una `request failed`, può proseguire interpellando il nodo corrispondente alla seconda chiave più vicina
- se il nodo non ha più modo di proseguire nella query, o ha già visto la richiesta, restituisce un messaggio di `request failed`

Se al termine della ricerca il file viene trovato, ogni nodo della catena si occupa di inoltrare la richiesta al mittente, memorizzando il file nel suo spazio di archiviazione e creando una nuova entry nella sua tabella di routing, associando la chiave al possessore del file. Si viene a creare un sistema simile ad una cache. In questa maniera richieste successive allo stesso file saranno sempre più rapide. I nodi finiscono per specializzarsi in maniera autonoma, memorizzando i file più richiesti.

Poiché le routing table hanno una dimensione limitata, pian piano le chiavi associate a file meno richiesti verranno soppiantate da quelle più comuni.

Per rendere più complessi possibili attacchi all'anonimato della rete, ogni nodo può scegliere autonomamente se memorizzare il file o addirittura agire come se non avesse un file che in effetti possiede.

Ingresso di nuovi nodi

L'ingresso di nuovi nodi è un punto cardine di ogni sistema P2P. Un nuovo nodo si mette in contatto con un nodo che fa già parte della rete. Per far sì che sia rintracciabile dagli altri nodi, il nuovo arrivato utilizzerà una cella di memoria dummy a cui associare una chiave da comunicare ai nodi adiacenti. In questa maniera, quando viene effettuata una query, sarà possibile rintracciarlo con il meccanismo visto in precedenza e rendere nota la sua esistenza, anche se inizialmente non offre nessun file utile.

L'ingresso di un nuovo nodo nella rete è strettamente monitorato, al fine di penalizzare parecchio i nuovi nodi fino a quando non si sono guadagnati la fiducia dei peer.

Anonimato

L'anonimato della rete viene ottenuto tramite la scelta eccentrica di inserire in ogni decisione di routing, un "rumore di fondo" probabilistico. Alcune decisioni apparentemente contrarie al buon senso del protocollo potrebbero essere intraprese con una probabilità non nulla, al fine di trovare uno stato migliore della rete, se questo esiste. In questa maniera, però, diventa praticamente impossibile stabilire con certezza il percorso di una richiesta e il mittente originale.

Chord

Distributed hash table

Un servizio scalabile P2P per il lookup di servizi attraverso internet, schematizzato da un articolo di ricerca prima della reale implementazione. Il protocollo si concentra nel trovare un modo per tradurre un indirizzo logico in un indirizzo IP che è possibile contattare.

Chord si definisce

- **efficiente:** $O(\log_2(n))$ per effettuare il lookup
- **scalabile:** $O(\log_2(n))$ informazioni da registrare in ogni nodo
- **alta performance** anche con variazioni repentine nel numero di nodi

dove n è la dimensione dello spazio di indirizzamento, ad esempio $n = 2^{160}$.

Tuttavia non presenta delle precauzioni nei confronti degli utenti malevoli, rendendolo soggetto ad attacchi DDOs o simili.

Funzionalità

Vengono implementate le funzioni:

- `insert(key, value)`
- `lookup(key)`

- `update(key, value)`
- `join(n)`
- `leave()`

Identificatori

Tutte le chiavi sono ottenute tramite l'applicazione della funzione hash SHA-1 alla risorsa o persino all'indirizzo IP del nodo che si vuole cercare. Gli indirizzi, o ChordID, possono essere visti come disposti su un cerchio, da leggere in senso orario.

Una delle regole principali del protocollo è che una risorsa con ChordID x verrà gestita dal nodo con l'indirizzo tale che il suo ChordID sia il primo immediatamente successivo sulla circonferenza.

Routing

La tabella di routing nel protocollo di Chord prende il nome di **finger table** ed è composta da m elementi. Questo perché ogni nodo conosce direttamente l'indirizzo di altri m nodi dell'anello con $2^m = N$. Ognuno dei loro indirizzi teorici sono posizionati ad un offset di 2^i $1 \leq i < m$ rispetto al proprio indirizzo sul cerchio dei ChordID. Tuttavia, poiché i nodi non ricopriranno tutti i possibili buchi del cerchio, la routing table si riferirà all'indirizzo successivo più vicino a quello teorico calcolato a partire dal ChordID. Nel momento in cui una risorsa viene richiesta, si procede domandandola al nodo che corrisponde al ChordID più grande fra quelli minori della chiave di indirizzamento della risorsa. Il nodo interpellato ripeterà la richiesta fino a trovare la risorsa cercata. Si tratta di una ricerca dicotomica esponenziale, che permette di ridurre la complessità ad una logaritmica.

Ricerca

La ricerca di una risorsa può essere riassunta dal seguente algoritmo:

- il nodo n vuole trovare la risorsa k
- n consulta la sua finger table e delega la richiesta al nodo con più distante da sé stesso con indice $i_1 < k$

- il processo si ripete nuovamente fino a quando un il nodo n_i presenta sulla sua finger table solo nodi con indice superiore a k . In quel caso il nodo più vicino è anche il nodo che gestisce la risorsa

Ingresso di nuovi nodi

Nel momento in cui un nuovo nodo si vuole inserire nella rete, deve scoprire a quali nodi comunicare la sua esistenza e quali nodi faranno parte della propria tabella.

Popolare la propria tabella di routing è banale, in quanto è sufficiente riempirla con i valori ideali per poi ottenere i valori reali semplicemente eseguendo la query per ognuno di essi.

Al contrario, aggiornare i propri predecessori può essere fatto se il nodo richiede alla rete la risorsa con l'id immediatamente precedente al proprio. In questo modo il nodo che risponderà sarà esattamente il predecessore.

Per assicurarsi che lo stato della finger table sia ancora aggiornato, periodicamente viene effettuata un'operazione di stabilizzazione, lanciando una serie di ping sulla rete.

Uscita di un nodo

Uscire dalla rete deve essere fatto con l'operazione apposita, dopo averlo annunciato sia al successore, al quale verranno delegate le risorse del nodo uscente, che al predecessore, che dovrà aggiornare la propria routing table.

Se il nodo crolla senza aver notificato chi di dovere, è probabile che si verifichino dei fallimenti nelle ricerche successive di altri nodi. Per evitare che l'intero sistema si blocchi, è previsto che ogni nodo conosca r successori del nodo in questione che replicano la stessa risorsa. Nel caso di un fallimento, la ricerca procede interpellando il primo successore valido ed aggiornando la finger table di conseguenza.

Il valore di r può essere scelto arbitrariamente alto per prevenire la perdita, anche se temporanea, di reperibilità dei dati.

Pastry

Distributed Hash Table

Pastry effettua un routing a livello applicativo in uno spazio di indirizzamento potenzialmente molto grande. Si occupa solo del sistema di routing, prestandosi a un qualsiasi sistema di distribuzioni, download e upload di risorse.

Il sistema è molto simile a quello di Chord, con uno spazio nell'ordine di 2^{128} rappresentato tramite una circonferenza da percorrere in senso orario. La differenza è la possibilità di usare basi diverse per rappresentare gli ID dei nodi in maniera più compatta.

Routing

Ogni nodo presenta ben tre tabelle di routing.

Leaf set

Contiene i nodi numericamente più vicini al nodo, divisi in peer successivi e precedenti rispetto alla rappresentazione circolare dello spazio di indirizzamento.

Neighborhood set

Contiene i nodi fisicamente vicini al nodo. La distanza potrebbe essere calcolata usando la latenza ad un ping.

Routing table

La tabella di routing vera e propria contiene b colonne e r righe, dove b è la base scelta per la rappresentazione degli identificativi, mentre r è il numero di cifre che compone ogni identificativo.

In ogni riga vi sono $b - 1$ id di altri nodi, con la $i - 1$ -esima cifra che assume tutti i possibili valori nella base b . Nella riga i -esima sono presenti gli ID dei nodi noti che hanno un identificativo che abbia i primi $i - 1$ bit in comune con l'ID del nodo che possiede la routing table.

Operando in questa maniera, si ottiene una suddivisione ricorsiva dello spazio di indirizzamento in b sezioni, che assicura la convergenza logaritmica dell'algoritmo di ricerca.

Vi sono diverse possibilità per cercare una route fino alla risorsa cercata.

- Si parte dalla routing table, interrogando i nodi più vicini alla risorsa.
- Se la risorsa non viene trovata, si va ad interrogare i leaf set.
- Se anche questa operazione non è fortunata, si vanno ad interpellare i neighborhood nodes.

Ingresso di nuovi nodi

Un nodo che vuole entrare deve contattare un nodo già presente nella rete e inserire il suo indirizzo in tutte le tabelle. Ogni volta che viene coinvolto nelle operazioni di ricerca si occupa di riempire tutte le entry della tabella.

Kademlia

Distributed Hash Table

Un sistema per il routing P2P basato su metriche XOR. Si tratta del sistema di routing alla base di implementazioni di molti client di criptovalute come Bitcoin. Questo lo rende uno degli schemi P2P più affermati di sempre.

È in grado di offrire parecchie feature che non erano riuscite ad essere risolte da sistemi precedenti, riuscendo a minimizzare il numero di messaggi di configurazione affinché un nodo conosca i suoi vicini. Le informazioni vengono acquisite e aggiornate come side-effect dei lookup. È anche possibile inviare più richieste in parallelo ottimizzando i tempi di attesa.

I nodi hanno abbastanza conoscenze da trovare dei percorsi a poca latenza. Vengono usate diverse richieste parallele asincrone per evitare dei timeout a causa di nodi caduti.

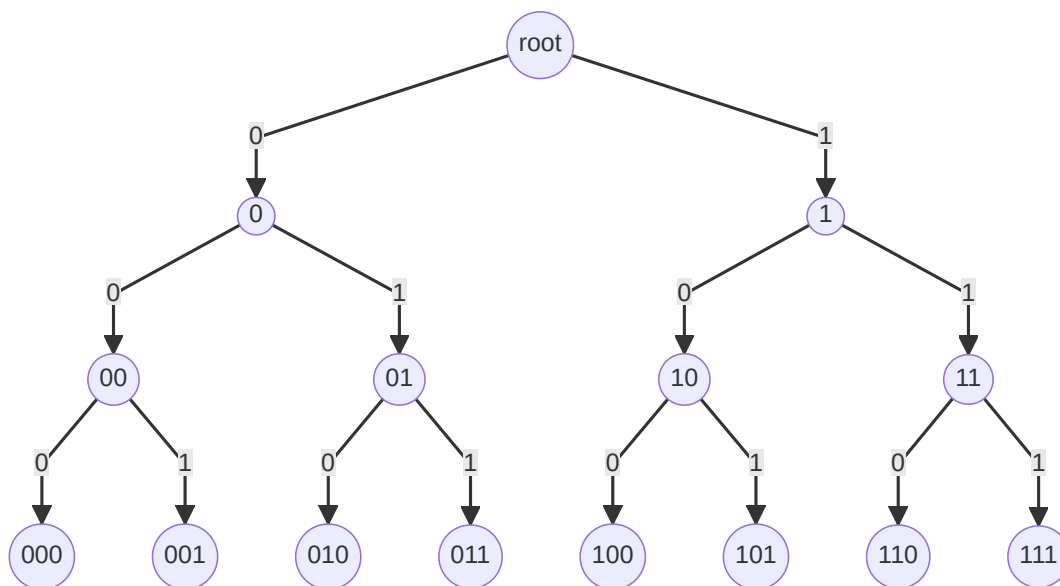
I percorsi vengono costruiti tenendo conto della latenza dal nodo contattato. Entrare a far parte delle tabelle di routing di altri nodi è un processo che richiede tempo, al fine di scoraggiare malintenzionati appena entrati nella rete.

Identificativi

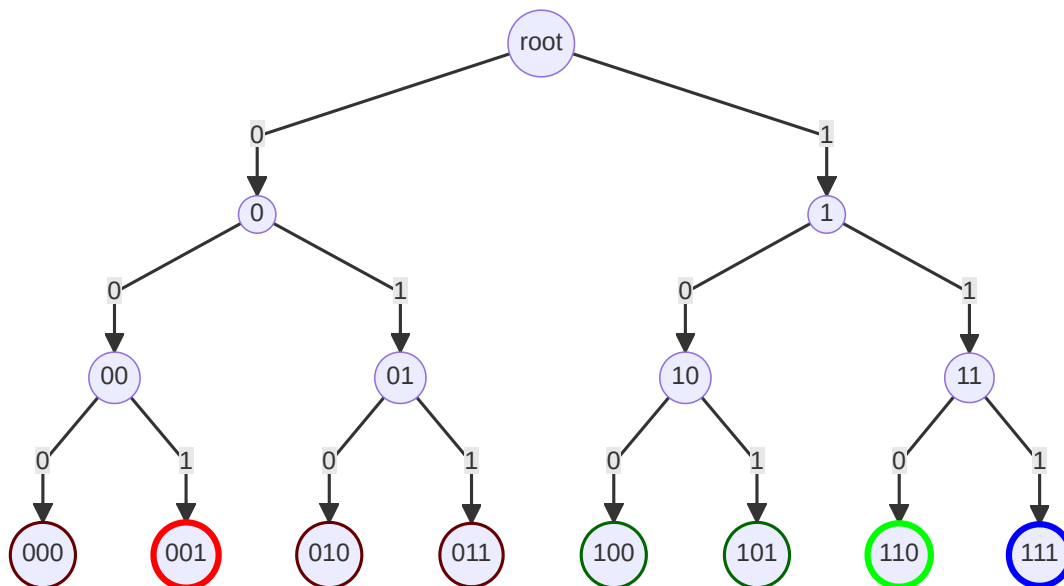
Tutte le risorse nella rete sono identificate dal loro hash prodotto con SHA1 di 160 bit. La distanza fra due risorse viene calcolata come lo XOR tra i due identificativi. In questa maniera, la distanza ha la proprietà di essere simmetrica. Quindi $\text{dist}(a, b) = \text{dist}(b, a)$.

Trie

Il routing di Kademlia è strutturato tramite un Trie. Si tratta di una struttura dati simile ad un albero, in cui il percorso per raggiungere la foglia rappresenta il suo identificativo. Si può quindi parlare di un albero binario di ricerca (nel caso la codifica sia in base 2) dove ogni arco che collega due nodi rappresenta una cifra. Il percorso fino ad una foglia va a formare il suo identificativo.



Poiché sia risorse che nodi ricevono lo stesso tipo di identificativo e fanno quindi parte dello stesso trie, è necessario stabilire una regola per capire come a quale nodo associare la risorsa. L'euristica scelta è quella del **più piccolo antenato comune**. Si risale l'albero livello per livello fino a che una risorsa non ancora assegnata incontra un nodo. Quel nodo sarà ora il suo responsabile.



Sopra è possibile vedere come le risorse (i nodi con i bordi più sottili) sono assegnate ai rispettivi nodi (i nodi con i bordi più spessi).

Per risolvere il caso ambiguo dell'ultima coppia di nodi, si è scelto di codificare questa stessa metrica applicando il concetto di distanza tramite XOR. Scegliendo infatti il nodo il cui identificativo è a distanza minima da quello della risorsa, si rende l'euristica esplicita e facile da computare.

K-Bucket

Ogni nodo inserisce i riferimenti ad altri nodi che conosce in una tabella composta da i righe, chiamate **k bucket**, con $i \in [0, k)$. Ogni riga contiene al massimo k elementi, con $k \in [0, 160)$.

I bucket sono organizzati in base alla distanza dal nodo che lo popola. Il bucket di indice i contiene solo i peer con distanza $2^i \leq \text{dist}(n, \hat{n}) < 2^{i+1}$ dove n è il nodo che popola il bucket e $\hat{n}$ è un altro peer che rispetta la disequazione.

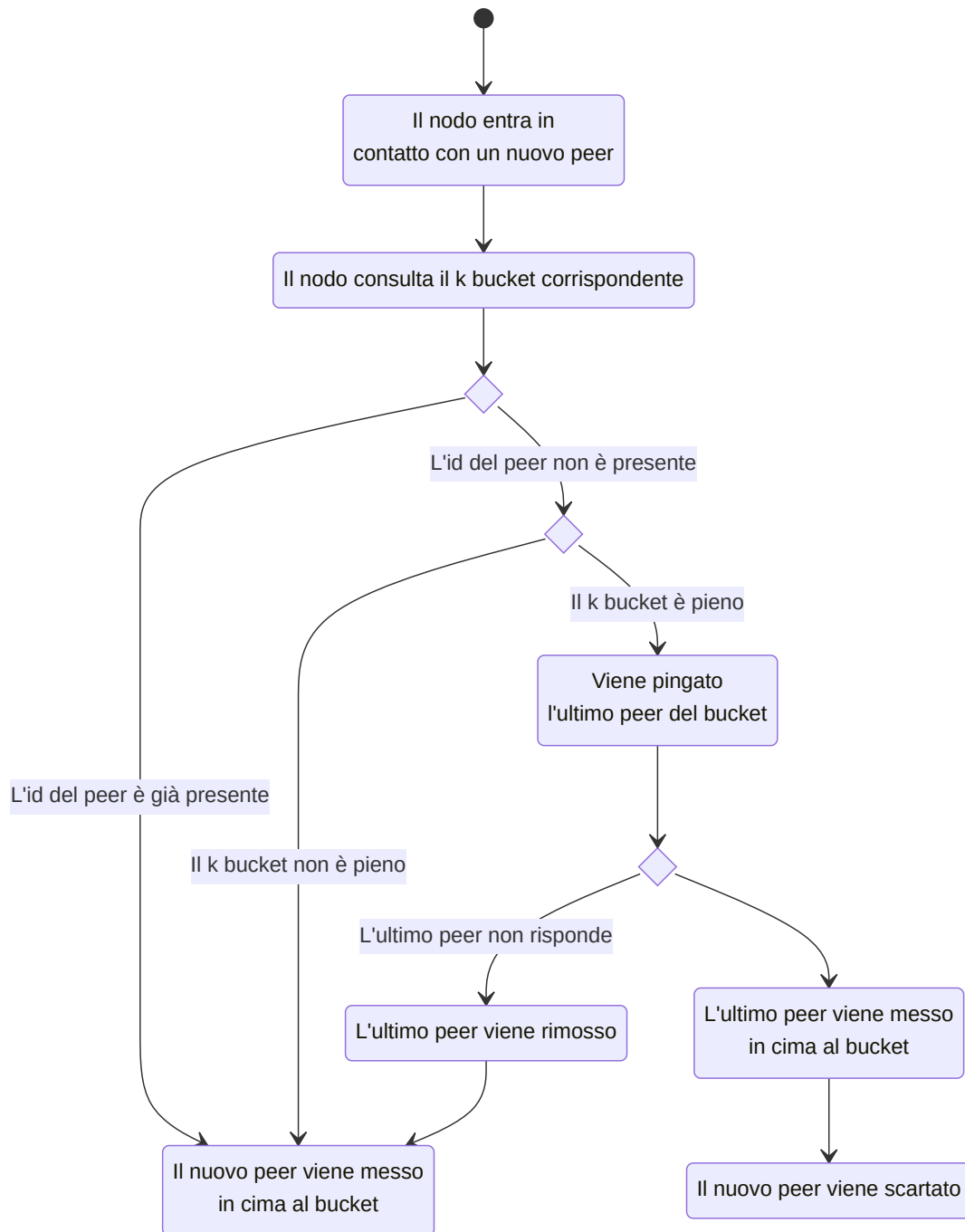
Ovviamente non è possibile inserire tutti i possibili nodi in ogni bucket (basta considerare che l'ultimo conterrebbe metà dei nodi dell'intero spazio di indirizzamento). Per questo viene imposta la dimensione massima k che i bucket possono raggiungere. Eventuali nuovi contatti vanno a rimpiazzare quelli più vecchi.

Ogni contatto nel bucket è formato da una tripla:

- il node ID, rappresentato in big endian
- il suo indirizzo IP
- la porta dove indirizzare la connessione UDP del protocollo

Aggiornamento del k bucket

Se un nodo riceve una comunicazione da parte di un peer, viene applicato il seguente algoritmo, che potrebbe aggiornare la tabella di routing.



Messaggi

- **PING:** controllare lo stato di un nodo
- **STORE:** ordina ad un nodo di memorizzare dei dati associati ad una chiave

- **FIND_NODE**: prende in input l'id a 160 bit. Il ricevente ritorna una tripla (ip, porta, id nodo) dei k nodi più vicini all'id richiesto, basandosi sul proprio k -bucket, se è pieno, altrimenti integrando aggiungendo peer da bucket di altri livelli
- **FIND_VALUE**: si comporta come un **FIND_NODE**. Se il ricevente ha eseguito una **STORE** per quel valore, lo restituisce.

Parallelismo nella ricerca

Una tecnica utilizzata dal protocollo per migliorare le prestazioni della ricerca è quella di eseguire la stessa operazione in parallelo comunicando contemporaneamente con più peer. Di fatto la stessa operazione di **FIND_NODE** o **FIND_VALUE** viene richiesta agli α peer più vicini all'id della risorsa o del nodo che si sta cercando. Il parametro α può essere aggiustato a piacimento, ma di default è 3.

I peer interpellati restituiranno la risorsa richiesta, se la conoscono. Altrimenti invieranno al mittente una lista di k nodi presi dalle proprie tabelle di routing con gli id più vicini alla risorsa cercata. Dopo aver ricevuto le risposte, se l'obiettivo non è ancora stato raggiunto, il mittente originale può proseguire la sua ricerca ripartendo dai risultati della query precedente.

Ingresso di nuovi nodi

Un nodo entrante dovrebbe avere il riferimento di un nodo già presente nella rete. Se non possiede già un identificativo, ne genera uno. Dopo di che chiede al nodo che conosce di cercare il proprio identificativo. In questa maniera, in cascata, il nuovo nodo si fa conoscere da tutti i nodi che verranno interrogati nella richiesta.

Inserimento di un file

Per memorizzare un dato all'interno della rete, un nodo effettua prima una ricerca sull'ID del file stesso. Dopo di che contatta k nodi ottenuti tramite la query ed invia a ciascuno di essi la richiesta di store. In questa maniera il dato è replicato fra tutti i k nodi.

Esiste inoltre un meccanismo di ripubblicazione, per cui dopo un intervallo di tempo prefissato, generalmente 24 ore, il mittente originale si assicura che la risorsa sia ancora disponibile effettuando nuovamente l'inserimento della risorsa. Se qualcuno dei nodi originali non fa più parte della rete, la risorsa verrà

nuovamente replicata.

Tabelle riassuntive

Sistemi ibridi: Napster

Pregi	Difetti
Semplice, in quanto la maggior parte della complessità è delegata al server	Facile da buttare giù a causa del server centrale coordinatore
Efficiente nel coordinamento e mantenimento della rete	La scalabilità è limitata dalle caratteristiche tecniche del server
Reperibilità delle risorse garantita dal server, a patto che i client non barino	Non garantisce l'anonimato
	Fiducia cieca nel server

Sistemi non strutturati: Gnutella 0.4, Freenet

	Gnutella 0.4	Freenet
Identificativi	MD4 del file	Chiave KSK, SSK, CHK
Ricerca	Tramite flooding fino ad esaurimento del TTL	Utilizzo di tabelle di routing che associano una chiave ad un nodo, altrimenti flooding fino all'esaurimento degli HTL
Performance	I messaggi di mantenimento della rete crescono in maniera esponenziale e saturano la banda	La ricerca può essere più lenta del necessario a causa delle scelte casuali: un nodo potrebbe affermare di non avere la risorsa pur possedendola
Anonimato	La rete è completamente distribuita, ma potrebbe essere possibile ricostruire il percorso dei messaggi	A causa delle decisioni di routing probabilistiche, è impossibile avere la certezza del percorso di un pacchetto nella rete

Sistemi strutturati: Gnutella 0.6, Gnutella 2, FastTrack, OpenFT, Emule, BitTorrent

	Gnutella 0.6	Gnutella 2	Emule	BitTorrent
Suddivisione dei peer	Ultrapeer e nodi foglia	Hub e nodi foglia	Client e Server	Tracker, Seeder e peer
Decentralizzazione	Gli ultra peer sono connessi fra di loro tramite una overlay network	Gli hub sono connessi fra di loro tramite una overlay network	Modello ibrido composto da due layer divisi fra server e client	Il tracker tiene traccia di tutti i nodi che possiedono la risorsa (seeder) e quelli che la richiedono
Architettura	Ogni nodo foglia è connesso ad un ultra peer. Gli ultra peer gestiscono 30 foglie	Ogni nodo foglia è connesso a due hub. Gli hub gestiscono da 300 a 500 foglie	Il server fornisce direttamente l'ubicazione del file nella rete	Il tracker indica ad un nuovo partecipante un sottoinsieme di peer interessati alla stessa risorsa con cui formare uno swarm
Identificativi	MD4 per i file	MD4 per i file	Low ID e High ID per i nodi, MD4 per file	File .torrent

	Gnutella 0.6	Gnutella 2	Emule	BitTorrent
Ricerca	La foglia invia la richiesta all'ultrapeer. Questo controlla il proprio filtro di bloom	La foglia invia la richiesta all'hub, che la invia solo a quelli adiacenti. Gli hub chiedono alle foglie che gestiscono	Download diretto dalla location indicata dal server	Il seeder suddivide il file in blocchi e lo distribuisce ai nodi dello swarm. Questi iniziano a scambiarsi tra di loro fino al completamento dello stesso, dando priorità ai nodi più generosi
Performance di routing	Trova la risorsa se nota agli ultrapeer	Trova la risorsa se nota agli hub	Trova la risorsa se popolare	Trova la risorsa se questa è popolare
Connessione	Fra due nodi foglia	Fra due nodi foglia	Fra due nodi	Fra due partecipanti dello swarm
Trasferimento	Chunk del file	Chunk del file	Chunk del file	Blocco ricevuto dagli altri peer dello swarm. Possibilità di choking

	Gnutella 0.6	Gnutella 2	Emule	BitTorrent
Reputazione	Sotto il controllo dell'ultrapeer	Per fare una richiesta la foglia deve possedere una query key firmata dall'hub	La coda di upload viene ordinata in base alla reputazione dei nodi riceventi. Questa dipende dal rapporto fra i dati caricati su quelli scaricati. Possono esserci altri modificatori come la permanenza nella coda ed eventuali ban	Vengono favoriti i nodi che condividono più blocchi e di maggiore rarità
Anonimato	Gli ultrapeer conoscono il traffico delle foglie	Gli hub conoscono il traffico delle foglie	Gli hub conoscono il traffico dei client	Lo swarm ha un qualche controllo sugli altri peer, e il tracker è noto pubblicamente
Point of failure	La rete di ultrapeer	La rete di hub	La rete di server	Tracker

DHT: Chord, Pastry, Kademlia

	Chord	Pastry	Kademlia
Spazio di indirizzamento	ID distribuiti in maniera circolare e unidirezionale	ID distribuiti in maniera circolare	Trie con foglie che rappresentano gli ID
Dimensione dello spazio di indirizzamento	2^{160}	2^{128}	2^{160}
Suddivisione dello spazio	Potenze di 2 fino a metà dello spazio	Quadranti via via più piccoli, in funzione della base scelta	Sottoalberi
Tabella di routing	Finger table	Leaf set + Routing table + Neighborhood set	Lista di k bucket
Definizione di distanza	Differenza modulare	Differenza modulare	XOR
Performance del routing	$\log_2(n)$	$\log_b(n)$	$\log_b(n)$

Wireless

Cablato Vs Wireless

Il cablato è pensato per la comunicazione punto-punto, con un tasso di errore estremamente basso. L'approccio è bit-pipe, per cui si può immaginare che i bit arrivino in uno stream continuo.

La comunicazione wireless sono broadcast per natura, per cui ci devono essere dei protocolli per evitare interferenze fra gli agenti che intendono comunicare. Inoltre c'è un tasso di errore estremamente elevato a causa delle molte fonti di disturbo e dei problemi di assorbimento e interferenza del segnale. Tutto questo con l'ovvio, enorme, vantaggio, di non necessitare il filo.

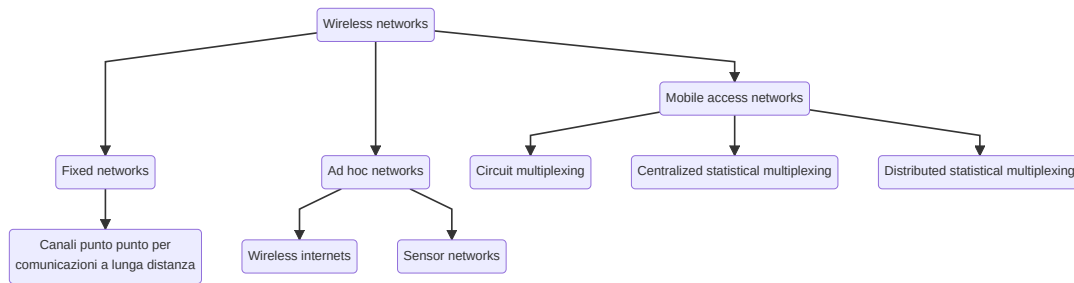
Le limitazioni imposte dal canale wireless comprendono una bassa larghezza di banda, con disconnessioni frequenti, reti frammentate ed eterogenee, con standard diversi e in continuo cambiamento. Anche la sicurezza viene messa alla prova a causa della natura broadcast della comunicazione.

La natura degli strumenti utilizzati nella comunicazione, perlopiù dispositivi portatili, impone di prendere in esame ulteriori questioni quali l'utilizzo energetico e la limitata potenza di calcolo.

Nella pila protocollare deve tenere conto delle peculiarità delle connessioni wireless:

- **Applicativo:** nuove applicazioni che si aspettano una connessione instabile
- **Trasporto:** il controllo di flusso e trasmissione deve essere rivista in termini di errori e perdita di pacchetti, non più dovuti principalmente alla congestione ma anche a fenomeni fisici riguardanti il mezzo di comunicazione
- **Network:** indirizzamento e routing
- **Data link:** accesso al mezzo e sincronizzazione fra i partecipanti
- **Fisico:** errori di trasmissione ed interferenze

Tassonomie



Livello fisico

I segnali si degradano esponenzialmente all'aumentare della distanza. Se le potenze dei segnali che si sovrappongono sono molto diverse, il più debole può apparire come rumore.

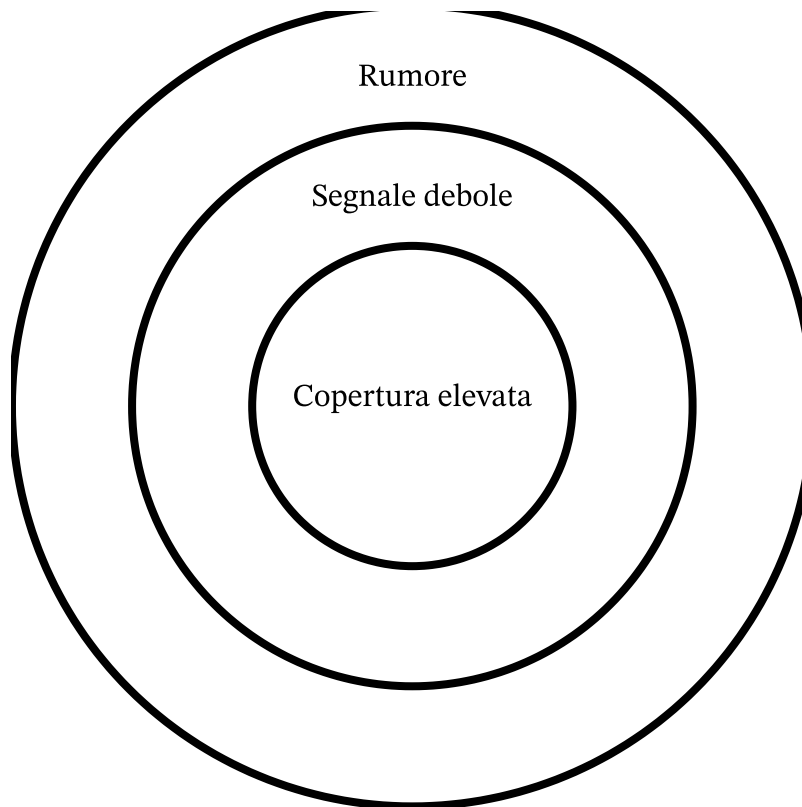
Ogni ricevitore ha una soglia sotto la quale è impossibile distinguere e decodificare un segnale.

La distanza dalla trasmittente gioca un ruolo fondamentale nel determinare la qualità del segnale.

L'auto-interferenza può assumere molte forme. Ad esempio, a causa della riflessione delle onde sulle varie superfici, si possono verificare interferenze o addirittura annullamenti dell'onda.

Raggio di ricezione e interferenza

In generale, a partire dalla fonte del segnale è possibile disegnare tre zone che descrivono come il segnale verrebbe recepito da eventuali ricevitori.



Le cose si complicano ulteriormente se, invece che semplificare utilizzando delle circonferenze, si considera il fatto che ogni antenna ha un proprio **diagramma di irradiazione**. Questo descrive l'emissione o ricezione del fronte d'onda dell'antenna nello spazio, specificandone l'intensità.

I tipi più comuni di antenne sono:

- **Dipolo:** propagano in tutte le direzioni, formando un cilindro
- **Pannello:** a forma romboidale, riescono a direzionare la copertura verso una zona specifica
- **Parabola:** utilizza la sua forma geometrica per direzionare tutti i segnali verso una piccola ricevente posta sul fuoco della parabola

Nei mezzi cablati il problema dell'interferenza prende il nome di collisione. Questo in quanto tutte le ampiezze sono comparabili, e nel caso due segnali si sovrappongono, non è più possibile estrarre i valori originali. Al contrario, per le comunicazioni wireless, distanza e potenza di base hanno un ruolo determinante per capire se i segnali sono in grado di interferire fra di loro o uno dei due ha un'ampiezza così bassa da sembrare rumore nell'altro segnale.

Tecniche di suddivisione del canale

Per evitare il problema delle interferenze, è necessario che l'accesso al canale sia suddiviso intelligentemente fra tutti i possibili trasmettitori.

Time-Division Multiplexing (TDM)

Il tempo di utilizzo del canale è suddiviso in **frame** della stessa durata. Ciascuno di questi frame è ulteriormente suddiviso in **slot**.

Ogni dispositivo ricetrasmittente ottiene a turno l'uso esclusivo del canale per un lasso di tempo prestabilito, prima di cederlo al prossimo utilizzatore.

Il problema di questo approccio è che necessita che tutti i dispositivi siano sincronizzati con precisione fra di loro.

Frequency Division Multiplexing (FDM)

Il canale di comunicazione viene suddiviso in sottocanali, ognuno costituito da una precisa banda di frequenze e separato da altri sottocanali da un piccolo intervallo di guardia.

In questa maniera, più dispositivi possono comunicare contemporaneamente senza provocare alcun'interferenza, fintanto che ognuno rimane all'interno della propria banda di frequenze.

Questa tecnica è ampiamente utilizzata nella radiofonia.

Code Division Multiplexing (CDM)

Ogni dispositivo possiede un codice univoco a 64 bit che utilizza per codificare tramite XOR il segnale che viene trasmesso. Così è possibile ricavare il messaggio originale del dispositivo di interesse dal segnale, anche in presenza di interferenze con altri segnali che seguono lo stesso protocollo, a patto di conoscerne il codice.

Per il ricevitore è sufficiente moltiplicare la sequenza ottenuta con il codice del mittente. Se la somma è prossima a 64, allora il risultato viene interpretato come 1, altrimenti 0. In altre parole, per trasmettere un bit ne vengono sprecati 63. Tuttavia si dà la possibilità di trasmettere contemporaneamente a 64 dispositivi.

Aloha e Slotted-Aloha

Si tratta di due protocolli storici studiati nell'ambito delle trasmissioni wireless.

Aloha	Slotted Aloha
Tutte le stazioni sono libere di trasmettere in qualsiasi momento	Tutte le stazioni possono trasmettere solo all'inizio di uno slot temporale
Il tempo non viene preso in considerazione	Il tempo è discretizzato in slot. Le stazioni devono avere una qualche forma di sincronizzazione
La massima efficienza teorica è del 18.4%	La massima efficienza teorica è del 36.8%

L'approccio di slotted aloha, nella versione definita **Random Access Channel (RAC)**, viene ripreso nella telefonia mobile con **GSM**. Finché i messaggi sono piccoli e poco frequenti rispetto alla dimensione del canale, questo sistema può funzionare.

Carrier Sense Multiple Access - Collision Avoidance (CSMA-CA)

Nell'ambito wireless non è possibile sperare di ottenere la Collision Detection. Questo perché trasmissione e ricezione del segnale operano a potenze estremamente diverse: il segnale emesso è molto più forte di quello che si riceve, e acceca il ricevitore dello stesso dispositivo. È quindi necessario cercare di prevenire le collisioni, perché nel momento in cui si trasmette non è possibile rilevarle.

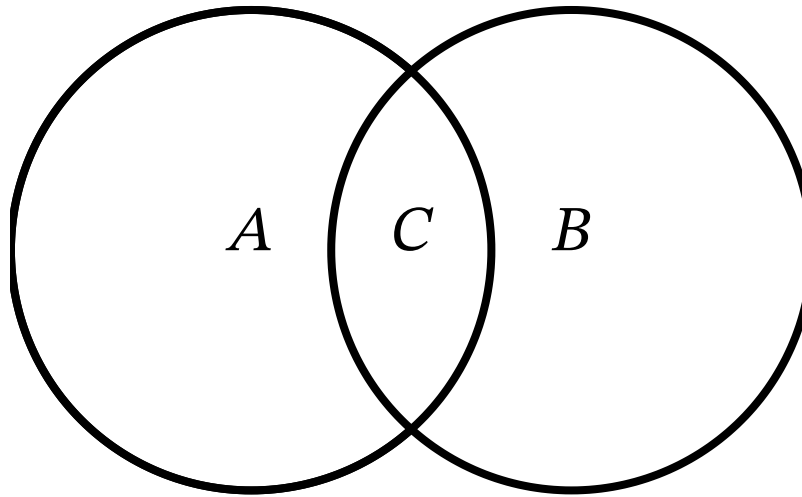
Il protocollo cerca di essere quanto più preciso possibile considerando due aspetti:

- la rilevazione della **portante fisica**
- definizione di una **portante virtuale**, con l'obiettivo di limitare le collisioni anche se la portante fisica non è percepita

La seconda portante cerca di evitare il problema del **terminale nascosto** (hidden node problem).

Terminale nascosto

Caso in cui un terminale centrale C riceve contemporaneamente una comunicazione da due nodi A e B che non sono in grado di percepirsi a vicenda. Si verifica una collisione su C



Terminale Esposto

Il primo nodo a comunicare nasconde con il proprio segnale quello dell'altro nodo.

Il protocollo IEEE802.11 utilizza proprio CSMA-CA.

Cellular wireless networks

GSM

Il cellulare riceve dalla BTS un frame di tempo ben preciso durante il quale può comunicare. Tuttavia, chiedere all'antenna il permesso di parlare richiede comunque una comunicazione. Viene quindi introdotto il Random Access Channel (RANCH). I messaggi supportati da questo sistema sono piccoli e poco frequenti in confronto alla capacità reale del RANCH.

La politica di accesso è quella di **ALOHA**. Tutti provano a comunicare immediatamente. Nel caso di una collisione si riprova dopo un ritardo casuale.

IEEE 802.11

Si tratta dello standard largamente adottato nell'ambito della comunicazione wireless.

L'uso dello spettro delle frequenze radio è regolamentato e ci sono solo delle frequenze ben precise che si possono utilizzare liberamente senza incorrere in penali. Si tratta degli intervalli $[2.4, 2.5]GHz$ e $[5.725, 5.875]GHz$, che vengono spesso utilizzate proprio dal Wi-Fi.

Per suddividere ulteriormente le bande libere, si usa la tecnica dello Spread Spectrum. Vi sono anche dei limiti imposti sulla potenza trasmessa: 1W sulle bande a 900MHz e 2.4GHz in America e 100mW in Europa. È generalmente già fissato dal produttore, ma potrebbe essere alterato dall'utente.

Livello fisico

Il livello fisico di IEEE 802.11 si suddivide in:

- **Physical Medium Dependent (PMD)**: definisce le caratteristiche e le metodologie di trasmissione e ricezione dei dati attraverso il mezzo radio tra due o più stazioni in base alla tecnologia usata (DSSS, FHSS, infrarossi, OFDM)
- **Physical Layer Convergence Protocol (PLCP)**: mappa i frame del MAC in frame adatti alla trasmissione secondo ciò che viene indicato dal PMD

Spread spectrum signal (SSS)

Le **spread spectrum signal** sono tecniche adoperate dai vari protocolli wi-fi. I dispositivi che adoperano il protocollo si riservano di occupare una banda maggiore rispetto a quella strettamente necessaria, ottenendo però, in cambio, una migliore ricezione dei segnali deboli, maggiore integrità e sicurezza nella ricezione del messaggio.

Il trucco è quello di distribuire il segnale su più frequenze, consentendo a più utenti di operare simultaneamente.

Frequency-hopping spread spectrum (FHSS)

Sistema inventato e brevettato da una coppia di ballerina e pianista.

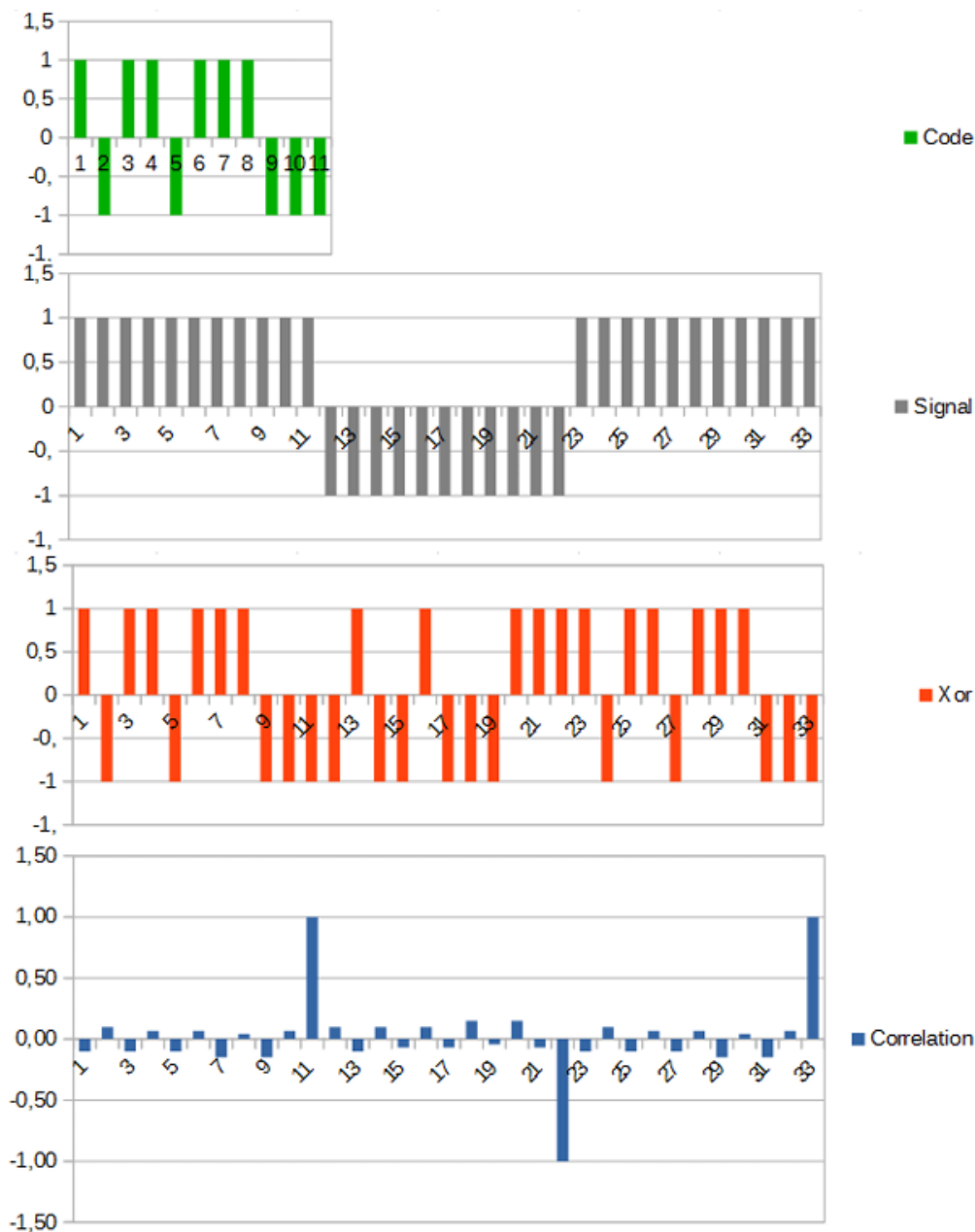
Il trasmettitore Wi-Fi è in grado di cambiare rapidamente e continuamente la frequenza a cui trasmette, in modo che un eventuale problema (un'interferenza o del jamming) su una specifica frequenza crei un danno limitato solo su quella frequenza, lasciando intatte tutte le altre.

Direct-sequence spread spectrum (DSSS)

Il **DSSS** è una tecnologia di trasmissione a banda larga, ogni bit viene trasmesso come una sequenza ridondante di bit, detta chip, con un approccio molto simile a **CDMA**. Tale metodo è indicato per la trasmissione e ricezione di segnali deboli.

L'interfaccia DSSS utilizza un codice di dispersione modulando il dato prima di trasmetterlo. Ogni bit trasmesso viene codificato con una sequenza a 11 bit.

Questo ha come conseguenza la diminuzione della larghezza di banda di 11 volte, ma rende la trasmissione molto più resistente al rumore del canale.



Configurazioni

IEEE 802.11 definisce ed è in grado di gestire diversi tipi di configurazioni dei nodi che compongono la rete wireless:

- **BSS:** un solo access point che organizza le comunicazioni cercando di evitare le interferenze. Stabilisce chi deve parlare e per quanto tempo
- **ESS:** più access point collegati tra loro attraverso una rete cablata (o comunque affidabile)
 - Il sistema di access point dell'università. I dispositivi possono trasferirsi da un access point ad un altro senza dover ripetere l'autenticazione

- **IBSS:** più access point che non sono in grado di coordinarsi attraverso un canale esterno
 - Avere un access point fisso non è sempre semplice o vantaggioso. È molto più comodo utilizzare delle reti ad hoc

Versioni di IEEE 802.11

- **802.11a:** opera sui 5-40 GHz raggiungendo velocità di trasmissione fra i 22 e i 54 Mbps. È un grado di fornire una copertura nel raggio di una ventina di metri.
- **802.11b:** opera a 2.4 GHz e ottiene una velocità di 11 Mbps, ma in cambio di una velocità più lenta, offre una copertura che può raggiungere un centinaio di metri.
- **802.11n:** introduce il supporto Multiple Input Multiple Output (MIMO) per migliorare la qualità del segnale, utilizzando un maggior numero di antenne di input ed output per dispositivo. Inoltre si appoggia anche sui canali con frequenze vicine

Un canale che lavora a 10Mbps è in grado di inviare le frame a 10Mbps. Tuttavia è necessario aspettare che la frame sia ricevuta, attendendo che il segnale si propaghi nello spazio. A questo si aggiunge il fatto che l'intestazione delle frame nella pila protocollare occupa uno spazio che riduce ulteriormente i bit utili trasmessi.

IEEE 802.11: prevenzione delle collisioni

Per evitare a tutti i costi il verificarsi di una collisione, il protocollo Wi-Fi implementa numerose misure di prevenzione. Appurato che non sia possibile realizzare un **physical carrier-sensing** come per i sistemi cablati, ci si accontenta di simularlo con un **virtual carrier-sensing**. Tale sistema è ottenuto tramite l'uso di appositi messaggi di gestione della rete che determinano il **Network Allocation Vector (NAV)**. Questo valore si traduce nella quantità di tempo durante la quale il canale sarà impegnato.

Distributed Coordinator Function (DCF)

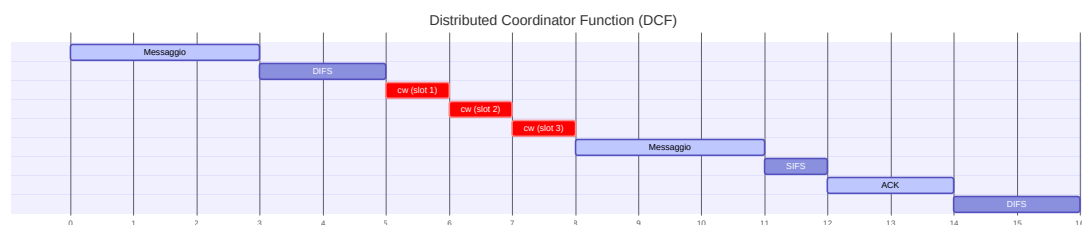
Si tratta di una implementazione di un protocollo per gestire l'accesso al mezzo fisico. L'accesso al mezzo fisico può essere diretto (e quindi conteso fra tutti i partecipanti) o mediato da un access point (PCF), che si occuperà di gestire la comunicazione con i vari dispositivi.

Una stazione che deve trasmettere ascolta il canale per un periodo pari (almeno) al tempo che intercorre fra due frame, chiamato **Distributed InterFrame Space (DIFS)**, che è almeno pari al tempo di propagazione del segnale nel canale. Se non vengono rilevate trasmissioni, la stazione può iniziare a trasmettere, altrimenti inizializza un timer detto **backoff timer**. Il timer viene bloccato durante la trasmissione di altre stazioni e viene riattivato dopo un intervallo di tempo superiore al DIFS successivo al momento in cui il canale è stato liberato.

Il timer è calcolato partendo da un intero $\in [0, cw - 1]$ dove cw è la finestra di contesa. Questa inizia alla fine del DIFS, per determinare chi deve trasmettere. Il tempo della finestra è divisa in slot. Una stazione può trasmettere quando il suo timer arriva a 0.

Questo sistema di CSMA/CA non è in grado di rilevare le collisioni. Una stazione che riceve correttamente una frame deve inviare un ACK di risposta. Dato che questo tipo di messaggi di controllo ha priorità, a loro viene applicato un timer più breve denominato **SIFS** ($SIFS < DIFS$).

Nel caso l'ACK non venga ricevuto, la frame viene considerata persa e il cw per quella stazione viene raddoppiato.



Protocollo MACAW

Il protocollo MACAW viene utilizzato per evitare i problemi di interferenza fra i segnali gestendo l'accesso al canale attraverso l'utilizzo di due messaggi di controllo: **Request To Send (RTS)** e **Clear To Send (CTS)**. La stazione che vuole trasmettere utilizza RTS per informare le stazioni nella sua zona di

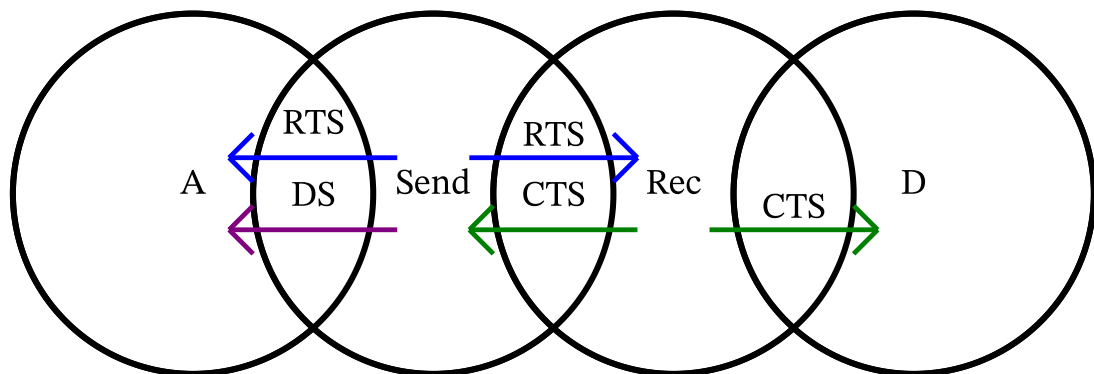
interferenza che ha intenzione di occupare il canale. Se il ricevitore è disponibile a ricevere, risponde con un CTS. Se invece ha già dato la propria disponibilità ad un'altra trasmittente, prima di inviare una nuova CTS aspetterà per assicurarsi che non ci siano collisioni.

Nel caso il protocollo sia partito correttamente, per cui il mittente ha ricevuto il CTS dal destinatario, allora invia ai nodi vicini un messaggio **Data Send (DS)** con lo scopo di informarli che la comunicazione sta partendo e di non utilizzare il canale.

Durante il periodo in cui il canale è bloccato, viene a crearsi un NAV. Agli occhi delle altre macchine, il canale è bloccato.

Se la trasmissione è sufficientemente lunga da richiederlo, la durata del NAV viene aggiornata con dei messaggi di ACK successivi che aggiornano la durata del NAV.

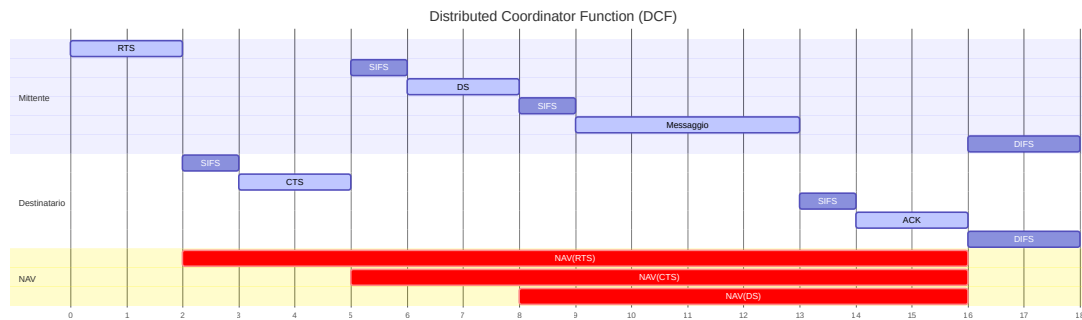
Lo scambio di RTS/CTS spreca molto tempo, per la latenza della comunicazione. La soglia della RTS può essere configurata, per evitare che frame di dimensioni inferiori a quelle impostate aspettino tutto il meccanismo RTS/CTS per essere inviate.



Facendo parte del protocollo MACAW e non dovendo più competere con gli altri dispositivi, tutti i messaggi dopo il primo RTS, quindi i CTS/DS e i dati scambiati dopo aver creato il canale virtuale, verranno inviati attendendo solo il tempo minimo (SIFS) fra una trasmissione e l'altra.

Dall'invio dell'RTS, il canale risulterà occupato per un tempo determinato dal campo NAV contenuto nella frame. Man mano che lo scambio di messaggi procede, questo valore potrebbe essere aggiornato, come nel caso in cui sia necessario proseguire nell'invio di messaggi.

Ovviamente il rispetto del NAV sarà limitato alle macchine che hanno percepito il suo invio. Tutte le altre continueranno ad utilizzare il canale indisturbate.



Tempi di attesa

Oltre ai tempi **SIFS** e **DIFS** visti precedentemente, esistono anche, dal più breve al più lungo:

- **RIFS**: tempo estremamente breve previsto da 802.11n utilizzato fra frame che farebbero parte dello stesso messaggio frammentato
- **SIFS**
- **PIFS**: utilizzato dagli access point
- **DIFS**
- **EIFS**: utile nel caso di una collisione. Quando vengono ricevute frame alterate a causa di un'interferenza, le macchine attendono un tempo maggiore di un DIFS prima del prossimo tentativo

Frame molto lunghe

Se la dimensione di una frame inizia ad essere particolarmente lunga, c'è la possibilità che il messaggio venga frammentato. Ogni frammento inviato attraverso il canale deve attendere l'ACK corrispondente.

Nel caso un ACK venga perso, viene avviata una procedura di backoff da parte del mittente.

CTS-to-self

Utile quando nella stessa rete coesistono macchine con protocolli diversi (velocità diversa, tempi SIFS, DIFS, ecc... diversi). La macchina, in genere l'AP, spedisce il messaggio CTS dove il mittente e il destinatario sono l'AP stesso. Serve per dire a tutti coloro che sono nella zona di copertura che non devono parlare. Questo CTS deve poter essere interpretato correttamente anche dai dispositivi più datati. Il canale viene riservato e si può proseguire con le velocità più elevate.

Compatibilità 802.11b e 802.11g

Le due versioni 802.11b e 802.11g utilizzano bitrate diversi rispettivamente 11 e 54 Mbps, per cui mantenere la compatibilità vuol dire abbassare lo standard più efficiente allo stesso livello di quello meno efficiente. In più è necessario utilizzare lo stesso

Compatibilità 802.11n

Un access point 802.11n può lavorare in 3 diverse modalità:

- **Legacy Mode:** supporto per la maggior parte delle versioni di Wi-Fi
- **Mixed Mode:** si prova ad adattare le performance in base all'interlocutore
- **Greenfield Mode:** si supportano solo i dispositivi moderni, migliorando parecchio le prestazioni

Throughput

In 802.11g viene utilizzato un sistema di modulazione **64QAM** che associa 6 bit ad ogni configurazione della portante. Utilizzando tutte le 48 sottoportanti si può arrivare ad avere 288 bit per configurazione. A causa degli errori molto frequenti, si utilizza un sistema di forward correction 3/4 che spreca un quarto della banda, arrivando a 216. Dato che la velocità di trasmissione è di 4 microsecondi, il throughput è di 54 Mbps.

In 802.11n vengono utilizzati ben 108 sottoportanti e un sistema di forward correction 5/6.

IEEE802.11: Point Coordinator Function (PCF)

Si tratta del modulo che fornisce un servizio al canale contention-free. Un nodo speciale, detto Point Coordinator (PC o AP) effettua il polling fra i nodi presenti, per scegliere l'unico a cui concedere il diritto di parola.

Una volta che l'access point ha avuto accesso al canale, spedisce un beacon frame, che contiene varie informazioni sulla configurazione della rete, sulla sicurezza e sulla propria identità. In una architettura BSS come questa, anche la sincronizzazione dei timer avviene tramite la frame beacon.

Se tutto va come previsto, dopo un intervallo PIFS, che ha maggiore priorità rispetto al DIFS, ma minore rispetto al SIFS, l'access point può inviare delle istruzioni per la gestione della comunicazione sul canale. Sulla beacon frame c'è anche il campo NAV che blocca il canale per tutta la durata necessaria.

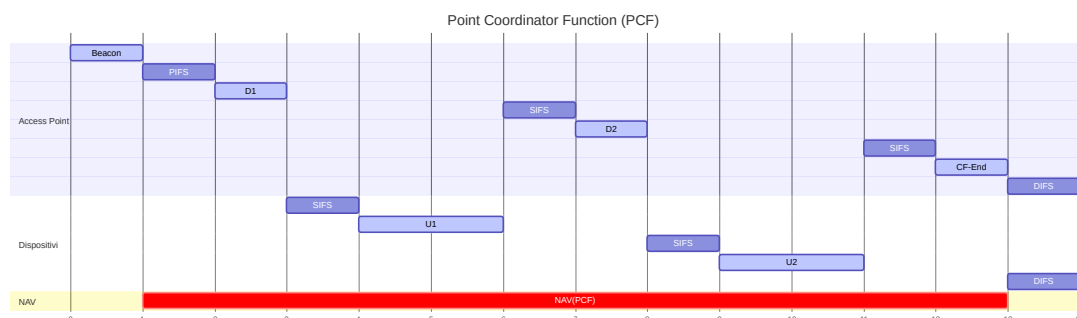
Per non occupare il canale a tempo indeterminato, alla fine della super frame proprietaria dell'access point, quest'ultimo invia una CFend, dopo la quale tutte le macchine possono iniziare a competere per il canale con tutti i protocolli e le modalità viste precedentemente.

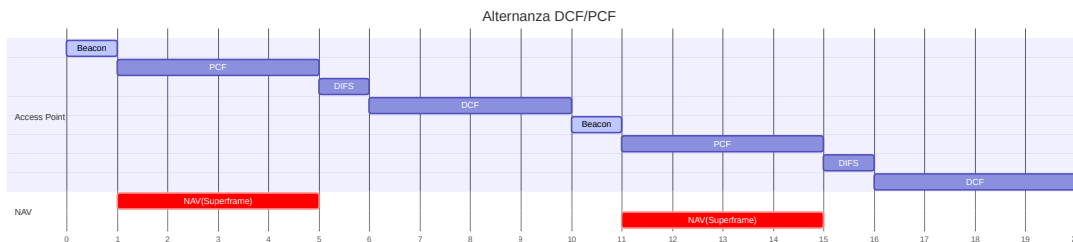
Questo dà spazio ad altri eventuali access point a stabilire la loro super frame e quindi il NAV che gli appartiene.

Il beacon interval spesso è un valore che può essere impostato.

Durante il periodo contention free è previsto che due macchine comunichino fra loro. Una volta ricevuto il consenso dall'access point, queste possono comunicare senza dover interpellare nuovamente l'access point.

Periodicamente il PC spedisce una frame di controllo, detta CF-Poll, per invitare nuove stazioni. La durata del periodo di contesa deve necessariamente essere tale da consentire l'invio di una frame di dimensione massima, comprese le necessarie frame di controllo.





IBSS

Per far funzionare il protocollo wireless in maniera distribuita, è necessario che ci sia una certa sincronizzazione per quanto riguarda il timestamp. Per far sì che sia sincronizzato, la stazione che vince la contesa, nella sua beacon frame inserirà un timestamp. Tutti gli altri access point che ascoltano la beacon frame controllano il timestamp e aggiornano il proprio se quello ascoltato ha un valore superiore al proprio. Eventualmente il timestamp più alto sarà condiviso fra tutti gli access point e quindi da tutte le macchine.

Se una nuova macchina vuole unirsi alla IBSS, è necessario che sincronizzi il proprio TFS timer con quello già in uso nella IBSS. Per inserirsi può:

- fare scanning passivo ed aspettare di leggere un beacon frame che contenga un timestamp da impostare
- eseguire uno scanning attivo, inviando una **Probe frame** e leggendo la conseguente **Probe Response**, spedita solo dal nodo che ha generato l'ultima beacon frame

Formato frame

```
[Frame control] [Duration NAV] [Addr 1] [Addr 2] [Addr 3] [Seq] [Addr 4] [Frame body 2 - 2312 bytes] [CRC]
```

Il campo duration NAV indica la durata della finestra temporale durante la quale il canale è riservato al mittente.

La frame prevede 4 indirizzi: mittente, destinatario, transmitter, source

I flag toDS e fromDS indicano il tipo di canale e la presenza (o assenza) di Access Point. Se sono entrambi posti a 0, non vi è access point. In caso contrario indicano come interpretare i quattro indirizzi precedenti.

Ci sono alcune frame speciali di controllo che sono molto più corte, come

- RTS [Frame control] [Duration] [Receiver address] [Transmitter address] [CRC]
- CTS [Frame control] [Duration] [Receiver address] [CRC]
- ACK [Frame control] [Duration] [Receiver address] [CRC]

Protocolli di routing

I protocolli tradizionali di routing si aspettano una stabilità piuttosto affidabile del grafo che forma la rete, al fine di poter ottimizzare la scelta del cammino in base alla brevità. Questo obiettivo viene raggiunto creando una tabella di routing che ogni nodo segue al fine di trovare il percorso migliore per l'instradamento del pacchetto.

Tale scelta non può essere applicata per quanto riguarda le reti wireless. Infatti, la struttura della rete cambia continuamente e molto di frequente. Senza contare che i nodi che compongono la rete hanno una limitata capacità energetica che si cerca di favorire scegliendo percorsi più stabili piuttosto che necessariamente i più veloci.

Approcci di routing

Vi sono diversi approcci che è possibile intraprendere nelle reti wireless. L'obiettivo è trovare il compromesso fra una corrispondenza aggiornata fra la tabella di routing e la situazione reale e un consumo di risorse e overhead in termini di banda sprecata quanto più piccolo possibile.

Approccio proattivo

- approccio più tradizionale per un percorso minimo distribuito
- deve aggiornare i percorsi costantemente, con un grosso overhead
- example: DSDV

Approccio reattivo

- determina il percorso solo quando necessario
- la sorgente avvia l'aggiornamento dei percorsi
- example: DSR

Approcci ibridi

- adattivi, cercando di combinare approcci proattivi e reattivi
- example: ZRP

Mobile IP

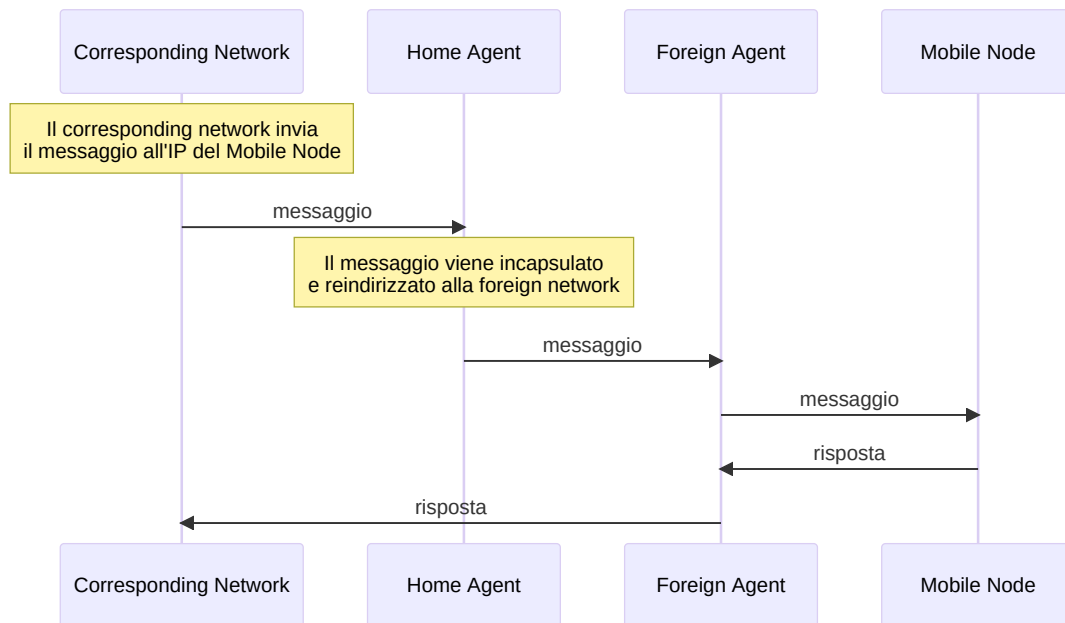
Il routing tradizionale è basato sullo schema di indirizzi IPv4. La possibilità di muovere liberamente i dispositivi rende necessaria la gestione del cambio di indirizzo nel caso di movimenti del dispositivo o la gestione di apposite tabelle di indirizzamento.

Cambiare indirizzo ip frequentemente necessiterebbe un DNS che viene aggiornato con la stessa frequenza. Inoltre cambiare l'indirizzo si traduce in interrompere regolarmente le connessioni TCP e la sicurezza può essere compromessa.

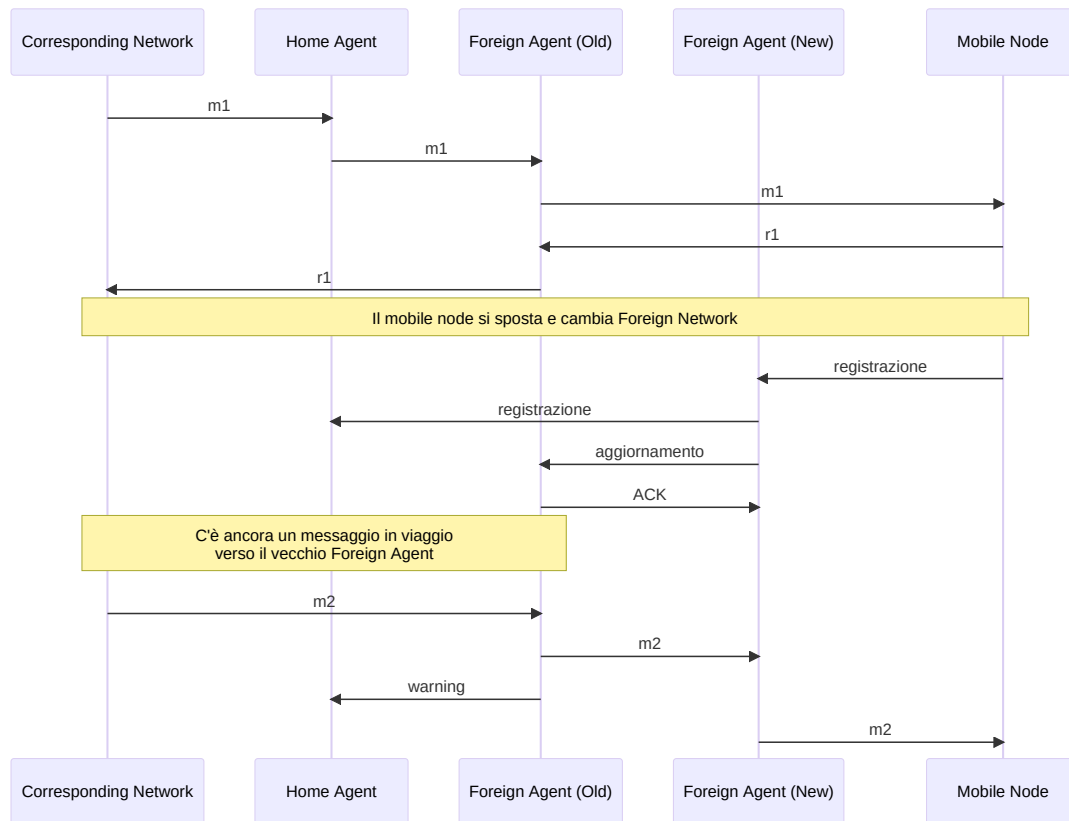
La soluzione è quella di dare al dispositivo mobile un IP fisso, che verrà incapsulato da un altro pacchetto con un IP tradizionale. Si parla infatti di IP over IP.

Vengono individuate più reti:

- **Home network:** rete a cui appartiene l'indirizzo IP posseduto attualmente dal noto mobile. Nel momento in cui intercetta un pacchetto che dovrebbe appartenere alla sua rete, lo rigira al destinatario corretto, impacchettandolo con l'indirizzo IP necessario a raggiungere la sua destinazione.
- **Foreign Network:** quando riceve un pacchetto che contiene un Mobile IP, invece di rifiutarlo, lo spacchetta e lo indirizza al dispositivo mobile interno
- **Corresponding Network:** tutto il procedimento è trasparente al mittente



Nel caso ci sia un cambiamento di Foreign Agent, il nuovo agente informa anche il vecchio agente, in maniera che se quest'ultimo riceve dei pacchetti rivolti al mobile node, sarà sua responsabilità inoltrarlo al nuovo agente, e comunicare un warning all'home agent.



Reverse tunneling

Sebbene normalmente non dovrebbero sorgere problemi nel routing della risposta che il Mobile Node invia al Corresponding Network, potrebbe capitare che, a causa di regole di firewall, un pacchetto con IP non corrette topologicamente venga scartato. Per evitare il problema, è possibile percorrere la strada al contrario ed instradare il pacchetto all'home agent, che si occuperà poi di inviarlo al vero destinatario. Di fatto si crea un tunnel IP anche nella direzione di ritorno.

Routing per reti wireless: approccio reattivo

Dynamic Source Routing (DSR)

Il protocollo è piuttosto vecchio, per cui non rispetta necessariamente tutti gli aspetti del protocollo Wi-Fi. Vengono inoltre aggiunte determinate ipotesi per far sì che tutto funzioni correttamente:

- i nodi vogliono collaborare
- il diametro della rete è di pochi salti
- i nodi possono muoversi, ma la loro velocità è bassa in relazione al raggio di copertura
- i nodi possono abilitare la modalità promiscua in ricezione
- i nodi devono essere indirizzabili univocamente

Il processo di route discovery viene effettuato tramite flooding. Il nodo mittente invia un messaggio di **Route REquest (RREQ)**, specificando la destinazione. Nel percorso, ogni nodo aggiunge un proprio identificativo. In questa maniera, i nodi possono scartare messaggi già visti, e il nodo finale conoscerà l'intero percorso che ha condotto il messaggio fino a lui.

Nel DSR gli archi fra i nodi potrebbero essere unidirezionali. In quel caso, conclusasi la ricerca, il nodo destinazione conoscerà il percorso dalla sorgente a se stesso. Per poterlo comunicare alla sorgente, sarà proprio la destinazione ad avviare un processo di route discovery verso la sorgente, per comunicargli il risultato attraverso una **Route REply (RREP)**.

Inoltre tutti i nodi sul percorso interessato sono in grado di ascoltare le

trasmissioni ed aggiornare la propria tabella con delle informazioni, anche se potenzialmente parziali. In questa maniera, eventuali richieste per la stessa destinazione possono ricevere una risposta cachata da uno dei nodi intermedi.

Nel caso la comunicazione non abbia successo, perché magari il nodo destinatario si è spostato o non è più in grado di seguire il protocollo, viene generato un messaggio di route error. Se la tabella di routing prevede un percorso alternativo rispetto a quello tentato, si prova attraverso la nuova strada. Una volta esaurite tutte le possibilità, si procede effettuando una nuova route discovery per aggiornare i percorsi fino alla destinazione.

Ogni mittente ha un sending buffer che contiene tutti i messaggi da spedire che non hanno ancora un percorso noto nella tabella di routing.

Per ognuno di essi esiste un timestamp per scartare i messaggi troppo vecchi, associati quindi a nodi che non risultano raggiungibili.

I vantaggi di questo approccio sono:

- i nodi iniziano a comunicare solo quando si vuole inviare un messaggio. Nei momenti di idle non è necessario scambiare messaggi
- il route caching contribuisce a diminuire l'overhead della route discovery. un nodo intermedio che trova nella sua tabella di routing un percorso verso la destinazione, risponde con questo alla richiesta di route discovery.
- una singola richiesta di route discovery può determinare la scoperta di più percorsi alternativi

I contro sono:

- la dimensione dei pacchetti aumenta con le dimensioni della rete e quindi del massimo numero di salti fino alla destinazione
- il flooding può interessare anche nodi che non sarebbero interessati, causando uno spreco di risorse
- problema delle collisioni risolto tramite un ritardo casuale nell'inoltro di pacchetti

Ad hOc Distance Vector (AODV)

In AODV, invece di affidare la memorizzazione della route all'header del pacchetto, ci si aspetta che ogni nodo mantenga una tabella per cui ad ogni possibile destinatario è associato il nodo adiacente a cui inviare il pacchetto.

Il processo di Route Discovery funziona in maniera simile a DSR, con la differenza che invece di mantenere traccia dell'intero percorso, ogni nodo si accontenta di conoscere la destinazione del messaggio e il dispositivo dal quale sta ricevendo la trasmissione, associando le due informazioni. Implicitamente, quindi, viene dato per scontato che tutte le comunicazioni siano bidirezionali. Questo vale sia per RREQ che per RREP.

Ad ogni riga è associato anche un contatore che viene aumentato ogni qualvolta si verifica un aggiornamento della riga. In questa maniera, quando si verifica un route error, e la tabella viene modificata, i nodi che ricevono l'informazione della riga possono verificare che è avvenuto un cambiamento più recente e aggiornano la loro tabella.

È inoltre presente un contatore di salti, che rende possibile confrontare percorsi differenti con lo stesso destinatario, nonché la lista dei precursori, cioè tutti i nodi adiacenti al percorso della routing discovery. Questo perché si tratta di nodi che ottengono delle informazioni parziali sui percorsi, ma devono essere notificati nel caso il percorso abbia subito una modifica.

Per assicurarsi che i collegamenti memorizzati nella tabella di routing siano ancora attivi, il nodo potrebbe sondare i dispositivi vicini con messaggi di Hello. Nel caso il vicino non risponda più, il nodo invia un messaggio di **Route Error (RERR)**. Tutti i nodi adiacenti interessati alla destinazione resa adesso irraggiungibile si preoccuperanno di avviare una nuova route discovery.

Location Aided Routing (LAR)

Rispetto ad altri algoritmi di routing, si aggiunge anche l'informazione legata alla propria posizione, scoperta attraverso un sensore GPS. Nel tentare di instaurare una comunicazione con un altro nodo, si crea una zona in cui il destinatario dovrebbe trovarsi, che viene allargata in una request zone.

Il processo di discovery avviene sempre tramite flooding, ma solo i nodi potenzialmente interessati, in quanto all'interno della request zone, partecipano all'attività, riducendo parecchio le comunicazioni inutili fra i nodi.

Link reversal Algorithm (LRA)

Nel LRA i link potrebbero essere bidirezionali, ma l'algoritmo li utilizzerà in maniera unidirezionale. Viene creato un grafo aciclico direzionato, con il nodo di destinazione come unico sink.

Il grafo è aciclico, e tutti i nodi devono avere un nodo in uscita, ad eccezione del

sink.

Nel caso un nodo si trovi senza archi uscenti, inverte tutti i nodi entranti. Nel caso tale operazione dia vita ad un nodo con lo stesso problema, l'inversione viene ripetuta. Si procede fino a quando tutti i nodi, eccetto il sink, hanno un arco in uscita.

Il vantaggio di questo approccio è che è distribuita, in quanto non richiede una cabina di regia, ed è autoconfigurante. Il sistema fallisce se è presente un partizionamento.

Temporally Ordered Routing Algorithm (TORA)

L'algoritmo TORA sacrifica l'ottimalità dei percorsi per privilegiare la stabilità dei collegamenti, preferendo quelli che sono in piedi da più tempo. Si comporta come una variante del LRA.

Altri protocolli

- Power-Aware Routing: si basa su DSR. Assegna un peso a ciascun link in funzione dell'energia consumata per trasmettere un pacchetto su quel link
- Associativity-Based Routing: si utilizzano solo i link che si sono dimostrati affidabili per un certo periodo di tempo
- Signal Stability Based Adaptive Routing: vengono favoriti i link con forte stabilità del segnale

Routing per reti wireless: approccio proattivo

Destination Sequenced Distance Vector (DSDV)

Si tratta di un protocollo che si basa sull'algoritmo di Bellman Ford distribuito. Ogni nodo ha una tabella di routing che indica a quale vicino rivolgersi per inoltrare il messaggio alla destinazione desiderata.

Ogni riga della tabella contiene sia il next hop che il costo di un percorso, oltre ad un numero di sequenza.

La tabella viene periodicamente inviata ai nodi adiacenti e aggiornata. Poiché i numeri di sequenza vengono incrementati con ogni aggiornamento del

percorso, assicurandosi però di produrre un numero pari. Numeri maggiori sono indice di informazioni più recenti e quindi preferibili.

Se il nodo destinazione diventa irraggiungibile, il suo numero di sequenza viene incrementato di 1, facendolo divenire dispari.

Destinazione	Prossimo	Salti	Num. Seq.	Timestamp	Stable_data
Nodo di destinazione	Nodo che fa da gateway per il nodo di destinazione	Numero di salti fino alla destinazione	Numero di sequenza generato dalla destinazione	Creazione della riga	Puntatore ad un file che tiene traccia della stabilità del percorso

Questo protocollo soffre del problema delle fluttuazioni. Questo avviene

Il costo maggiore di mantenimento e aggiornamento della tabella rispetto ad altri protocolli alternativi, viene giustificato nel caso in cui sia necessario avere un percorso sempre pronto nel momento in cui è necessario inviare un messaggio e la dimensione della rete è contenuta.

D'altra parte, tutti i nodi devono partecipare continuamente al protocollo, senza poter entrare in idle, perché altrimenti perderebbero gli aggiornamenti e verrebbero esclusi dalla rete.

Clusterhead Gateway Switch Routing (CGSR)

Tutti i nodi sono raggruppati in cluster e fanno capo ad un **cluster head**, che si occupa di scoprire e contattare i nodi gateway. Si tratta di dispositivi in grado di mettere in comunicazione più cluster, poiché parte dell'intersezione degli stessi. All'interno del singolo cluster il protocollo di routing utilizzato è DSDV.

Optimized Link State Routing (OLSR)

Si basa sul link state. Ogni nodo però deve aver ben chiara la topologia della rete.

Si cerca di minimizzare il broadcast dei messaggi scegliendo dei nodi opportuni nella propagazione del messaggio. Questi nodi speciali si chiamano **Multirelay point**. I multirelay point di un nodo x sono l'insieme di tutti i nodi tali che tutti i nodi a 2 hop da x siano contemporaneamente adiacenti a un nodo multirelay point.

Tutti i nodi si scambiano le tabelle di routing che contengono i nodi adiacenti, così che tutti siano in grado di individuare il set di multirelay point.

Routing per reti wireless: approccio ibrido

Zone Routing Protocol (ZRP)

Si tratta di un protocollo che combina un approccio reattivo ad uno proattivo.

Tutti i nodi entro una distanza d dal nodo x sono nella **routing zone** di x .

I nodi a distanza esattamente pari a d sono detti nodi periferici della stessa routing zone.

Vengono distinte due categorie di nodi: quelli all'interno della routing zone e quelli fuori. Per i primi, si utilizza un approccio proattivo come DSDV, mentre per quelli esterni ci si accontenta di un approccio reattivo come DSR o AODV.

Questo tipo di protocollo si comporta particolarmente bene quando il traffico si concentra nelle zone interne.

Delay Tolerant Network (DTN)

Affinché internet per come lo conosciamo funzioni correttamente alcune condizioni fondamentali devono essere verificate.

- deve esistere almeno un percorso fra i due endpoint
- il round trip time deve essere limitato
- l'approccio principale è quello dello switch del pacchetto
- i protocolli di routing funzionano perfettamente se i collegamenti sono piuttosto stabili
- l'affidabilità sulla consegna del messaggio può essere ottenuta utilizzando gli appositi protocollo, come TCP

Le DTN cercano di aggirare alcune di queste limitazioni.

- in un determinato istante temporale, potrebbe non esistere un percorso fra mittente e destinatario
- i collegamenti potrebbero non essere sempre presenti
- vi sono grandi ritardi di propagazione
- ci sono diverse architetture di rete
- i protocolli di routing sono basati sull'approccio **memorizza, trasporta e inoltra** (store, carry and forward).
- non prevedendo un routing predefinito, la scelta del prossimo nodo a cui inviare il messaggio può cambiare in ogni momento

DTN vs MANET

Manet	DTN
Mobilità dei nodi	Mobilità dei nodi
Nessuna infrastruttura	Nessuna infrastruttura
Risorse limitate	Risorse limitate

Manet	DTN
Reti partizionabili	Reti partizionabili
Usa TCP/IP	Usa un protocollo bundle layer
Richiede un percorso end-to-end	Non richiede un percorso end-to-end
Best effort delivery	Store-Carry-Forward delivery
Tempo trascorso nel nodo minimo	Tempo trascorso nel nodo lungo

Caratteristiche delle DTN

Il partizionamento della rete è una delle caratteristiche chiave della DTN, mentre può rendere una destinazione irraggiungibile nel networking classico.

Ogni nodo DTN è in grado di memorizzare un pacchetto per parecchio tempo prima di inoltrarlo. Questo perché il fatto che i nodi si muovano è una parte integrante del protocollo.

Nel TCP tradizionale il TCP lavora end-to-end, e quindi tutta la parte di rete diventa trasparente al protocollo, e si suppone la comunicazione sia real time con ritardi minimi. Dato che tale ipotesi non è vera per le DTN, non si pretende di ricevere alcun ACK dal destinatario, salvo che dopo averlo richiesto espressamente.

Bundle layer

Si tratta di uno layer posizionato sopra il livello di trasporto. Un bundle consiste di tre parti principali:

- **header:** uno o più blocchi inseriti dall'agent
- **dati della risorsa,** incluse informazioni ottenute dall'applicazione dirette all'applicazione di destinazione. Potrebbe includere istruzioni su come processare o gestire i dati dell'utente in arrivo
- **bundle trailer:** zero o più blocchi DTN inseriti dal protocollo bundle

Routing

Routing opportunistico

Nel caso ci siano due end-point che vogliono comunicare fra di loro, non è detto che esista un collegamento in grado di trasportare il messaggio fra le due parti. Il protocollo deve quindi supportare la possibilità che il pacchetto aspetti un tempo anche piuttosto lungo prima di essere inviato al nodo successivo.

Come scegliere quando effettuare il salto e verso quale nodo?

Social based routing

- Bisogna stabilire una metrica che renda possibile stabilire numericamente il valore di ogni nodo.
- Diventano importanti anche il suo profilo/interessi ed utilizzarlo come tramite per comunicare un messaggio ad una destinazione con simili interessi.
- La storia delle interazioni contiene delle informazioni passate di un nodo e le utilizza nella selezione del prossimo nodo.
- Un approccio gerarchico utilizza delle gerarchie già presenti nella rete facendo risalire il messaggio di livello fino a raggiungere un nodo che ha modo di notificare il destinatario o un suo superiore.
- Un altro approccio è effettuare un flooding nella community del nodo di destinazione, con la speranza che il messaggio lo raggiunga disturbando il minor numero possibile di nodi non interessati.
- Ci possono essere dei nodi appositamente dedicati a fare da tramite per altri nodi.
- Se ci sono dei raggruppamenti in base a cluster è necessario raggiungere il cluster del nodo di destinazione
- Si effettua una stima per ogni salto, determinando la possibilità che la distanza con l'obiettivo si accorci
- Il messaggio viene frammentato in maniera tale che il messaggio possa essere ricostruito con un sistema a soglia che necessiti solo di una piccola percentuale di frammenti. Fare in modo che i frammenti seguano percorsi differenti amplifica la possibilità che il destinatario riceva il messaggio

- Il flooding rimane la tecnica migliore. Idealmente, infatti, ci si assicura che il messaggio percorra tutte le possibili strade, ottenendo la probabilità maggiore possibile che la destinazione riceva il messaggio. D'altro canto, bisogna assicurarsi che questa operazione si arresti e vi è uno spreco ingente di risorse.

Epidemic

SI tratta di un protocollo di flooding che introduce un numero massimo di hop (TTL) e una quantità di memoria massima che è possibile utilizzare per il buffer dei messaggi. Giocando con queste due proprietà è possibile modificare la percentuale di messaggi che raggiungono la destinazione.

Spray and wait

Il routing viene diviso in due fasi principali.

- **spray**: il nodo originale effettua l'invio di l copie del messaggio in l canali differenti
- **wait**: se non si è ancora raggiunta la destinazione, gli l nodi procederanno ad inoltrare il messaggio senza utilizzare il flooding

Nella sua variante binary, il TTL viene modificato in maniera diversa per ogni nodo. Infatti, dopo aver ricevuto il messaggio, il nodo invierà due copie del messaggio. Se l'originale aveva un TTL di n , i due messaggi avranno TTL 2 , $n - 2$. Questo darà vita ad un albero non bilanciato.

ProPhet

Si tratta di un protocollo opportunistico che determina una probabilità di consegna basata sullo storico di incontri fra i nodi. Questo valore è calcolato come $P_{(A,B)} = P_{(A,B)_{old}} + (1 - P_{(A,B)_{old}}) \cdot P_{init}$ dove $P_{(A,B)}$ è la probabilità che il nodo A incontri il nodo B .

Poiché la formula è solo crescente, è necessario introdurre un fattore di invecchiamento, definita come $P_{(A,B)} = P_{(A,B)_{old}} \cdot \gamma^k$ dove $\gamma \in [0, 1)$ e k è il numero di unità di tempo trascorse dall'ultima volta che la era stato effettuato l'aggiornamento.

Per finire, il protocollo definisce anche una proprietà transitiva, per cui se il nodo B conosce il nodo C e A conosce B , il nodo A ottiene una probabilità di incontrare C in maniera transitiva attraverso B . Questa viene definita come

$P_{(A,C)} = P_{(A,C)_{old}} + (1 - P_{(A,C)_{old}}) \cdot P_{(A,B)} \cdot P_{(B,C)} \cdot \beta$. con $\beta \in [0, 1)$. Tale valore deve essere comunque sottoposta ad aging.

Nonostante tutto lo studio, ci sono casi in cui il protocollo non si comportava correttamente. Ci sono infatti dei problemi nel momento in cui il nodo deve decidere se inviare il messaggio al prossimo nodo. La scelta viene effettuata a seconda delle probabilità calcolate in precedenza. Tuttavia potrebbe capitare il caso in cui le condizioni sono cambiate e le probabilità storiche non riflettano più la possibilità del nodo di comunicare con la destinazione.

Rimane anche il dubbio sul numero di copie che il nodo di partenza invierà. Per stabilire questo valore potrebbe essere utile basarsi sulla probabilità dei nodi incontrati. Incontrare un nodo con probabilità 20% significherà inviare una copia e scalare un contatore inizializzato a 100 di 20. In maniera analoga si procede con nodi con probabilità diverse.

Un modo per ridurre il traffico della rete e le risorse richieste ai nodi potrebbe essere quello di introdurre una soglia minima per il trasferimento da un nodo ad un altro.

ProPhet inoltre non ha alcun riguardo per la privacy. Ottenere dei valori di probabilità sugli incontri consente al nodo di farsi un'idea piuttosto precisa dei movimenti degli altri nodi e un grafo delle interazioni fra nodi.

Per risolvere questo problema, si è migliorato il protocollo introducendo della cifratura omomorfica.

Si definisce il concetto di habitat del nodo, cioè le zone in cui è più probabile incontrare il nodo in questione. Questo concetto è stato ulteriormente semplificato indicando solo la zona in cui è probabile incontrare il nodo utilizzando solo un'ellissi. Al di fuori della figura, la probabilità è bassa, al suo interno la probabilità è superiore alla soglia stabilita.

Nel caso il nodo continui ad essere all'interno dell'habitat, tutto rimane invariato. Se invece il nodo esce, la figura viene aggiornata.

Per evitare di dover effettuare operazioni di esponenziazione con la cifratura omomorfica, necessarie per utilizzare la distanza euclidea, si utilizza la cosiddetta distanza di Manhattan (Taxicab geometry).

Per semplificare ulteriormente le operazioni, invece di utilizzare un'ellisse, si utilizza un rettangolo con gli spigoli smussati. La figura viene ulteriormente suddivisa in 9 sezioni.

Le sezioni in questione vengono utilizzate per stabilire se la destinazione sia all'interno di una delle zone note di uno dei nodi in cui si entra in contatto. Se sì, si tende a preferire le zone più piccole. Altrimenti si sceglie quella che si avvicina di più.

Varianti TCP

Le implementazioni di TCP sono molte, e usano algoritmi differenti per ottenere un controllo della congestione end-to-end.

- Tahoe
- Reno
- NewReno
- Vegas
- SACK
- FACK
- Westwood
- Veno

A causa della base installata enorme, è necessario che ogni versione, anche quella più nuova, sia compatibile con tutte quelle precedenti, per evitare che si verifichino problemi nella comunicazione fra le macchine.

Questo però non è vero per tutte le versioni. Mentre Tahoe e Reno sono assolutamente compatibili fra di loro, altre, come SACK, aggiunge dei dati che saranno ignorati da client che non supportano la stessa versione. Non vengono quindi sfruttate tutte le potenzialità del protocollo.

Problemi di TCP

Nelle prime versioni di TCP vi erano diverse problematiche in grado di comportare un uso molto inefficiente della rete.

Congestione

Un aumento del carico della rete che inizia a saturarla, provocando una riduzione delle informazioni utili che giungono a destinazione

Le cause della congestione possono essere molteplici:

- spreco di banda utile
 - alcuni protocolli, come telnet, necessitavano l'invio di 40 byte di intestazione per comunicare un singolo byte utile
- ritrasmissione inutili di pacchetti che stanno ancora girando nella rete
 - una possibile soluzione prevede dei timer più accurati e il controllo della congestione
- pacchetti non recapitati
 - controllo di congestione per tutto il traffico
- frammenti di pacchetti che, se persi, rendono necessario ritrasmettere l'intero pacchetto
 - evitare di frammentare i pacchetti
 - trovare un MTU che vada bene per tutta la rete
- controllo del traffico
 - una gran parte del traffico nella rete è prodotto da messaggi di controllo (Headers, routing, DNS, ...)
- pacchetti vecchi o non più interessanti
 - dato che i pacchetti subiscono un ritardo quando vengono memorizzati nelle code
 - conviene eliminare il prima possibile i pacchetti che verrebbero spediti dopo un ritardo inaccettabile

Per cercare di migliorare la gestione della congestione, TCP utilizza varie strategie:

- riuscire a trovare un equilibrio. Per non saturare immediatamente la rete, a discapito degli altri utenti, bisogna partire lentamente, incrementando pian piano la banda utilizzata
- una stima quanto più accurata possibile del timeout da applicare nel momento in cui si ritrasmette un pacchetto
- eliminare le ritrasmissioni inutili. Per farlo, è necessario avere un Round Trip Time (RTT) accurato

Sistema auto-sincronizzante o ACK clock

Si tratta di un sistema che si autoregola secondo gli ACK di risposta. Tende a raggiungere uno stato molto stabile anche in caso di ritardi. Un pacchetto può essere inviato solo in risposta ad un ACK del pacchetto precedente. L'unico momento delicato è l'avvia del sistema.

Il problema è che un sistema del genere funziona solo se si lavora con una singola coppia sorgente-ricevente e vi sono solo il buffer di invio e di ricezione.

Aggiungere dei buffer intermedi è sufficiente a rompere la stabilità del sistema ed impedirgli di essere in grado di autoregolarsi. Ogni macchina, infatti, può avere cognizione solo del proprio buffer e del canale su cui comunica, ma non conosce la situazione di altri canali e buffer.

La situazione ideale di massimo utilizzo della rete prevederebbe che tutti i canali siano pieni e i buffer vuoti. Nel caso di canali con prestazioni differenti, si fa riferimento al canale con la capacità inferiore.

Algoritmi TCP

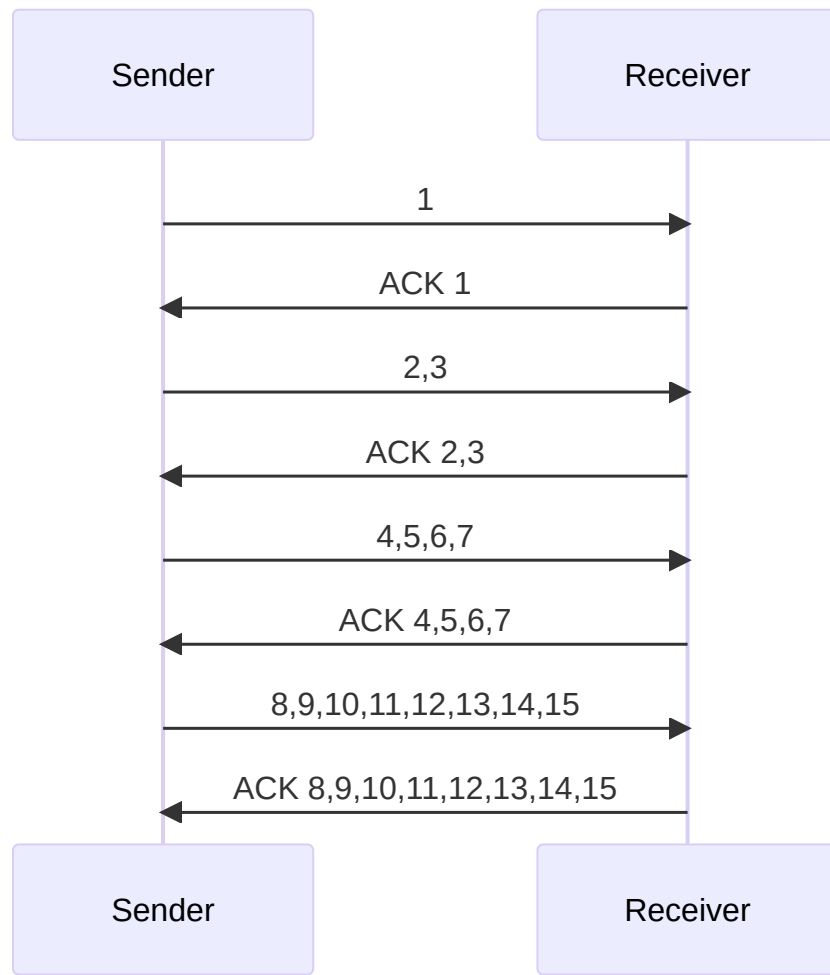
CI sono 4 algoritmi principali utilizzati dalle implementazioni più comuni di TCP.

Congestion window

Quantità di pacchetti inviati insieme nella rete.

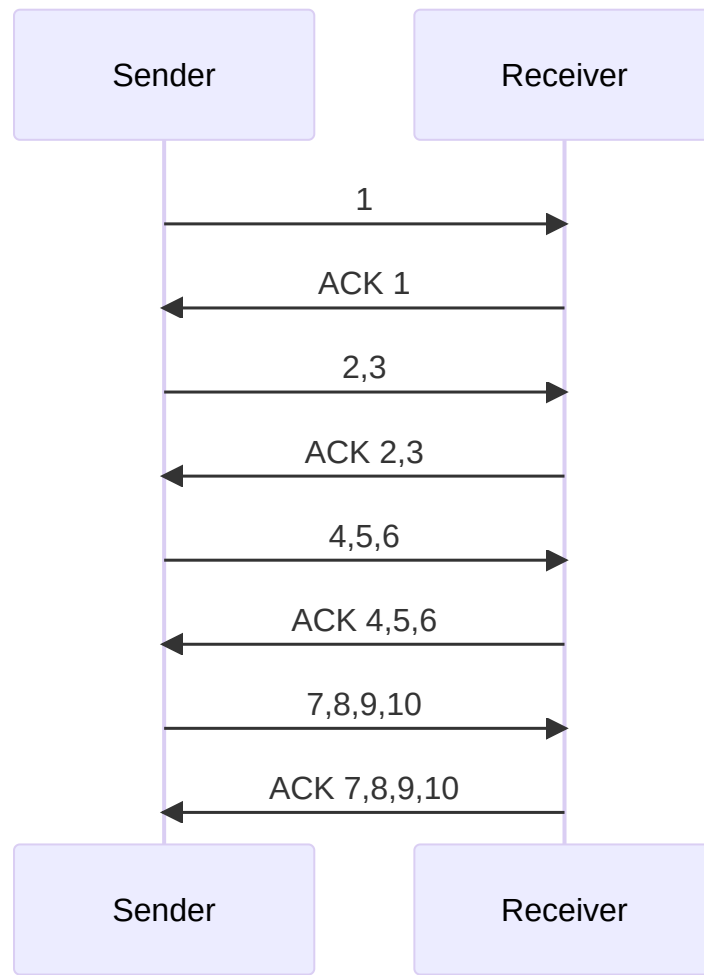
Slow start

Ogni volta che viene ricevuto un ACK, la congestion window viene incrementata di 1.



Congestion Avoidance

La congestion window viene incrementata di uno per ogni RTT. Più precisamente, la finestra aumenta come $cw = cw + \frac{1}{cw}$ ad ogni ACK.



Fast retransmit

Invece che attendere fino a far scattare il timeout del pacchetto, nel caso si ricevano 3 ACK duplicati si procede a ritrasmettere il pacchetto che, con ogni probabilità, è andato perso.

Fast recovery

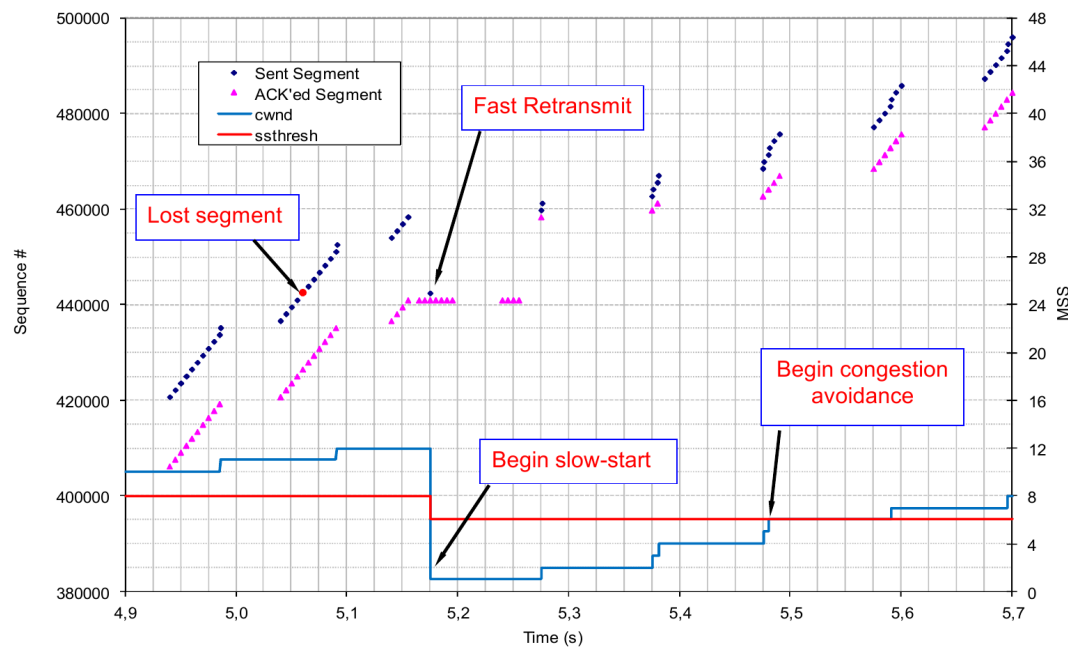
Nel momento in cui si è verificata la perdita di un pacchetto, e si sono quindi ricevuti 3 ACK duplicati, si procede a dimezzare la congestion window.

TCP Tahoe

Si tratta di uno delle primissime implementazioni di TCP.

Implementa lo **slow start**, la **congestion avoidance** e il **fast retransmit**. Non utilizza, però, il fast recovery.

Dopo la perdita di un pacchetto, anche se la criticità viene risolta dal fast retransmit, la congestion window viene comunque reimpostata ad 1 e di riparte con la slow start.



TCP Reno

TCP Reno nasce come un miglioramento rispetto a ciò che era stato fatto con Tahoe.

Vengono aggiunti l'**algoritmo di Nagle**, migliorata la stima dell'**RTO** e viene implementato il **fast recovery** dopo un fast retransmit.

Algoritmo di Nagle

Per migliorare le prestazioni del sistema è conveniente evitare di far partire pacchetti contenenti troppi pochi dati. Questo problema diventa piuttosto evidente nel momento in cui, dopo essere stato saturato, il buffer del ricevente inizia a svuotarsi, permettendo l'invio di nuovi dati da parte della sorgente. Se i dati venissero inviati il prima possibile, si verificherebbe il fenomeno noto come **silly window syndrome**, che riduce decisamente le prestazioni della comunicazione.

Prima di effettuare una spedizione, viene quindi consultato l'algoritmo di Nagle.

```

if data_to_send is not null:
    if window_size >= maximum_segment_size and buffer.length >=
maximum_segment_size:
        send(data_to_send)
    else:
        if unconfirmed_data in buffer:
            buffer.enqueue(data_to_send)
        else:
            send(data_to_send)

```

Retransmission Time Out (RTO)

Il tempo di timeout viene calcolato come una funzione dell'RTT.

Ottenere una stima dell'RTT dell'ultimo pacchetto si traduce nel misurare la differenza fra il tempo di arrivo dell'ACK e quello dell'invio del pacchetto: $\text{SampleRTT} = t_s - t_a$.

L'RTT medio verrà poi calcolato come una media pesata di più SampleRTT, secondo la formula $\text{EstimatedRTT} = \alpha \cdot \text{EstimatedRTT} + (1 - \alpha)\text{SampleRTT}$, con $\alpha \in [0.8, 0.9]$.

L'approccio proposto presenta un difetto notevole quando si lavora con pacchetti ritrasmessi, in quanto non è possibile stabilire a priori a quale pacchetto si riferisce l'ACK (l'originale o quello ritrasmesso).

Per evitare questo problema, **Karn/Partidge** propongono di evitare di aggiornare l'RTT quando si ha appena effettuato una ritrasmissione e di limitarsi ad aggiornare il tempo di timeout con il doppio del suo valore attuale.

La formula standard per la stima dell'RTT viene applicata nuovamente quando ci si ritrova in una situazione di regime.

Jacobson/Karels procederanno a rendere ancora più precisa la stima dell'RTT tenendo in considerazione anche la varianza del SampleRTT.

$$\begin{aligned}
 \Delta_{RTT} &= \text{SampleRTT} - \text{EstimatedRTT} \\
 \text{EstimatedRTT} &= \text{EstimatedRTT} + \beta \cdot \Delta_{RTT} \\
 \text{Deviazione} &= \text{Deviazione} + \beta \cdot (|\Delta_{RTT}| - \text{Deviazione}) \\
 &\text{con } \beta \in [0, 1]
 \end{aligned}$$

$$\begin{aligned}
 \text{Stima del timeout} &= \mu \cdot \text{EstimatedRTT} + \Phi \cdot \text{Deviazione} \\
 &\text{con } \mu = 1, \Phi = 4
 \end{aligned}$$

Se la varianza è piccola, il timeout è molto vicino alla stima dell'RTT. Se invece c'è una grande varianza, questa domina la formula.

Fast Retransmit & Fast recovery

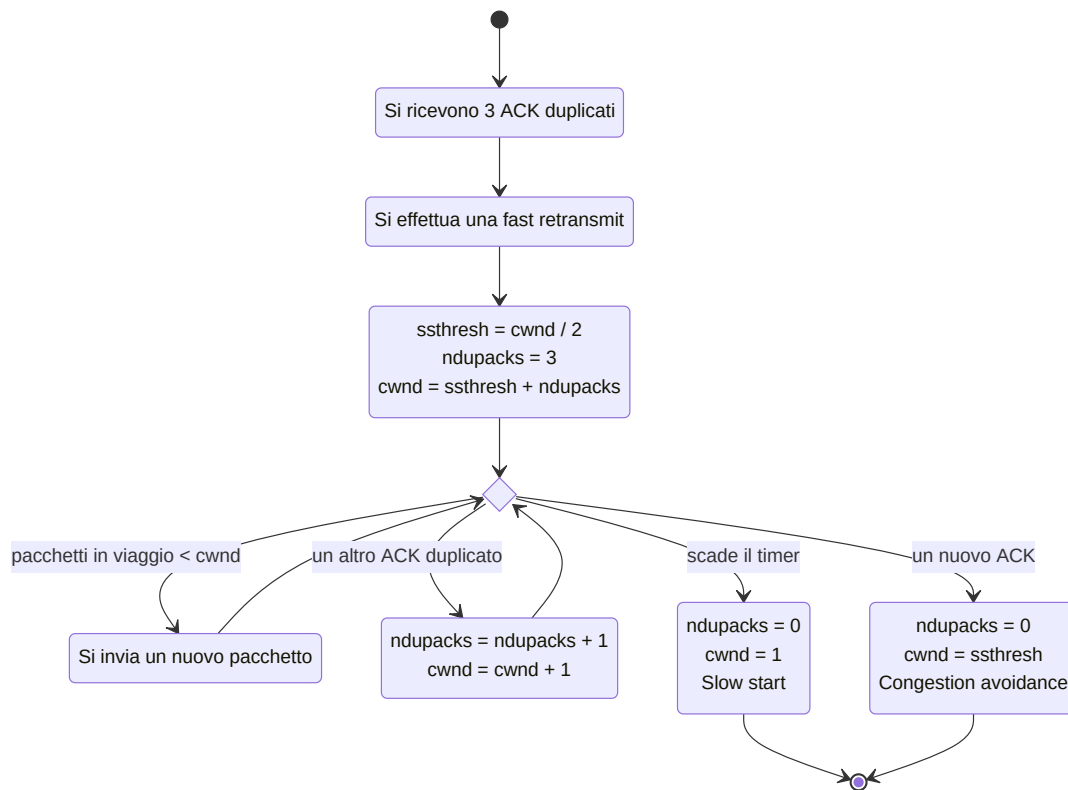
Dopo aver ricevuto 3 ACK duplicati

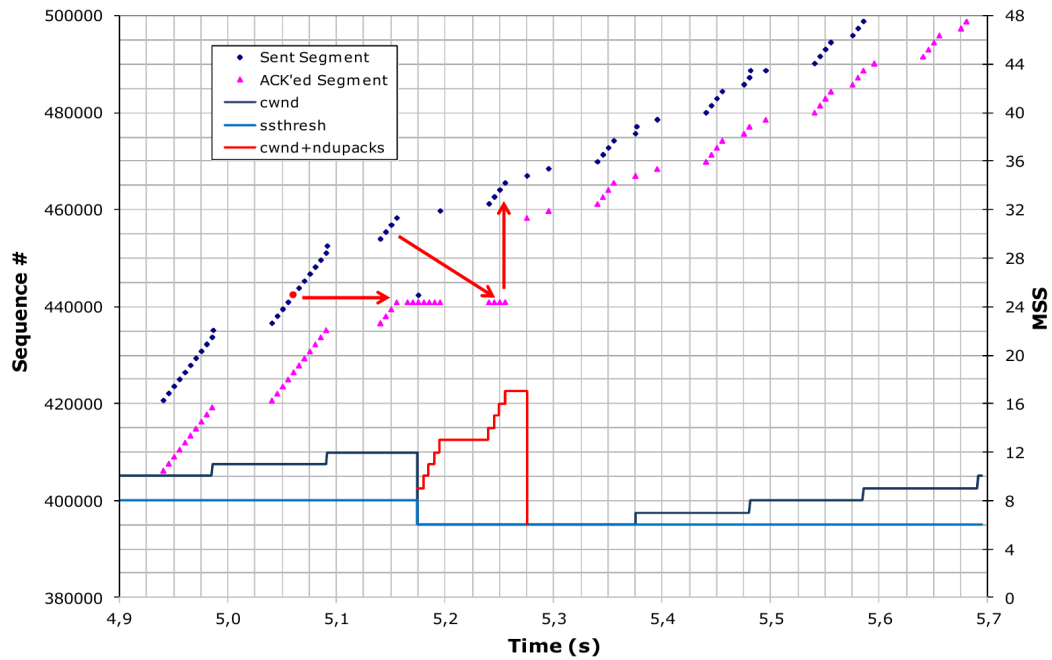
- il segmento perso viene ritrasmesso
- la `ssthresh` viene impostata a $cwnd/2$
- il `cwnd` viene impostato a $ssthresh + ndupacks$

Finché continuano ad arrivare ACK duplicati, si incrementa il `ndupacks`. Se il numero di pacchetti ancora non confermati diventa inferiore a `ndupacks`, si invia un nuovo pacchetto.

Non appena si riceve un ACK nuovo, il `ndupacks` viene azzerato e si esce dalla modalità di fast recovery.

Se il timer scade senza che la situazione si risolva, si azzerava il `ndupacks` e si effettua una slow start.

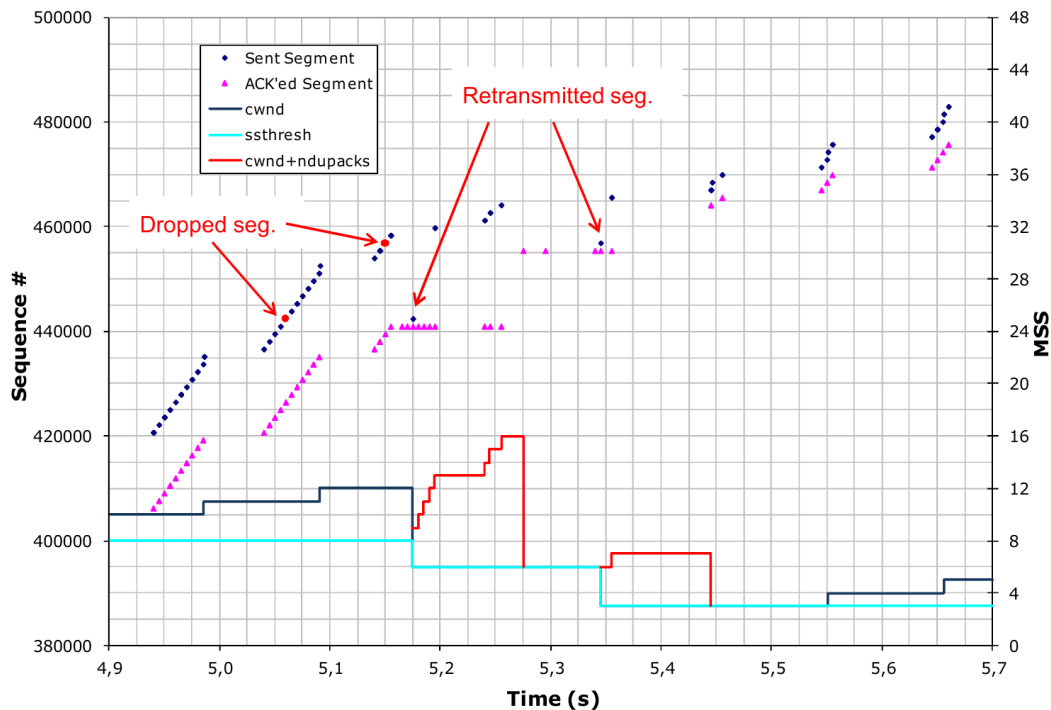




Slow start (Tahoe) vs Fast recovery (Reno)

Finché le perdite che causano la fast retransmit sono sporadiche, Reno tende a comportarsi meglio, rallentando meno di Tahoe. Se si raggiungono anche solo 4 perdite in poco tempo, però è quasi certo che Reno vada in timeout, peggiorando notevolmente le prestazioni rispetto a Tahoe.

Il problema nasce dal fatto che l'algoritmo di Reno non supporta bene delle perdite che avvengano nella fase di fast recovery.



TCP NewReno

NewReno nasce con lo scopo di migliorare le limitazioni di TCP Reno e renderlo quindi più affidabile anche nel caso di perdite multiple consecutive, apportando qualche modifica all'algoritmo di fast recovery.

Partial ACK

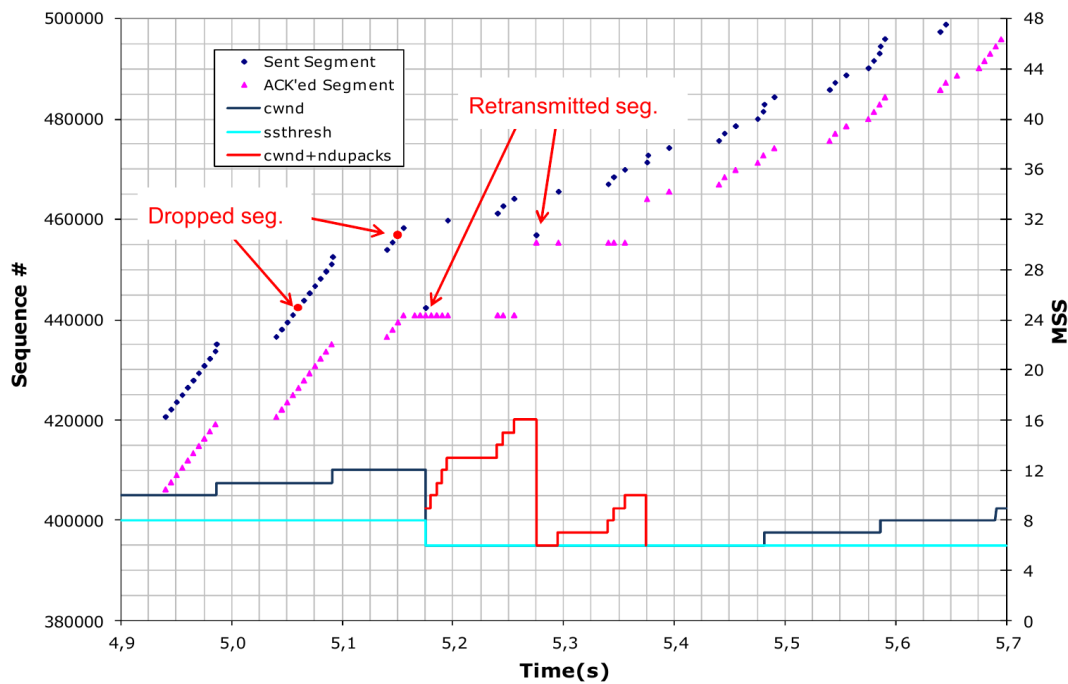
Si tratta di un ACK che fa riferimento ad un pacchetto precedente a quello atteso e che concluderebbe la fase di fast recovery.

Indica che solo una parte dei pacchetti ha raggiunto la destinazione. Si è verificata almeno un'altra perdita.

Se si riceve un partial ACK durante la fase di fast recovery

- si ritrasmette immediatamente il pacchetto perso
- si riduce la finestra di congestione rimuovendo i pacchetti che sono stati confermati dal partial ACK
- si aggiunge un `ndupacks`
- si invia un nuovo pacchetto, se le dimensioni della finestra di congestione lo consentono

Il mittente rimarrà in fase di fast recovery fino a quando tutti i pacchetti non confermati quando la fase stessa è iniziata non saranno stati confermati, per poi tornare in collision avoidance.



Reset del timer

Nella fase di fast recovery, il timer potrebbe essere resettato ad ogni partial ACK o solo al primo. I due approcci si distinguono in:

- **impaziente:** il timer per il timeout viene resettato solo dopo aver ricevuto il primo partial ACK. Al suo scadere, si riparte con la slow start. Ciò può accadere più facilmente se la perdita riguarda più pacchetti.
- **lento e paziente:** il timer per il timeout viene resettato ogni volta che si riceve un partial ACK. Questo permette al protocollo di rimanere nella fase di fast recovery anche per parecchio tempo, fintanto che si verifica qualche perdita

TCP Vegas

Viene introdotto da Brakmo and Peterson nel 1994 e presenta 3 principali differenze rispetto a TCP Reno:

- la congestion avoidance cerca di prevenire un timeout, e si attiva non appena il throughput reale risulta inferiore a quello atteso, percependo il formarsi di congestione
- la retransmission avviene al primo ACK duplicato, se la stima del RTT è maggiore del tempo di timeout

- lo slow start cerca di stabilizzarsi

Congestion avoidance

La stima dello stato della rete, e la sua potenziale congestione, viene fatta basandosi sulla differenza fra gli RTT stimati e quelli reali degli ultimi pacchetti.

$$\begin{aligned}\text{Expected throughput} &= \frac{WindowSize}{BaseRTT} \\ \text{Actual throughput} &= \frac{TransmittedBytes}{SampleRTT_{last}}\end{aligned}$$

$$\text{con } BaseRTT = \min\{SampleRTT_i\}$$

Si noti che

$$\begin{cases} BaseRTT \leq SampleRTT_{last} \\ TransmittedBytes \leq WindowSize \end{cases} \implies \text{Actual throughput} \leq \text{Expected throughput}$$

$$Diff = \text{Expected throughput} - \text{Actual throughput}$$

Si determinano due valori, α, β che corrispondono al numero di pacchetti che il protocollo è disposto a mantenere in coda. In base al confronto fra questi valori e Diff, il protocollo effettua la correzione della finestra di congestione e della stima del throughput atteso.

$$\begin{cases} \text{Banda sottoimpiegata} \implies \text{incrementa } cwnd & \text{se } Diff < \alpha \\ \text{Banda ben impiegata} & \text{se } \alpha \leq Diff \leq \beta \\ \text{Banda sovraimpiegata} \implies \text{decrementa } cwnd & \text{se } Diff > \beta \end{cases}$$

Cambio di route

Nel corso di una connessione, il percorso che i pacchetti seguono potrebbe cambiare.

- se la nuova route è più corta della precedente, il *SampleRTT* sarà più breve, e TCP Vegas continuerà ad operare correttamente
- se la nuova route è più lunga, il *SampleRTT* verrà incrementato. L'algoritmo però interpreterà il cambiamento come congestione sul canale, riducendo inutilmente il throughput

La soluzione adottata è quella di verificare se per N pacchetti consecutivi, $SampleRTT$ è sempre più grande di $BaseRTT$. Se ciò è vero, si aggiorna il valore di $BaseRTT$ e si incrementa la dimensione della finestra.

Congestione persistente

Se all'avvio di una connessione che utilizza TCP Vegas il canale è già congestionato, il $BaseRTT$ calcolato provocherà l'istanziamento di una finestra di dimensioni superiori a quelle corrette.

Per limitare il problema, una possibile soluzione è quella di utilizzare la **Random Early Detection (RED)** nei router per limitare la crescita delle code.

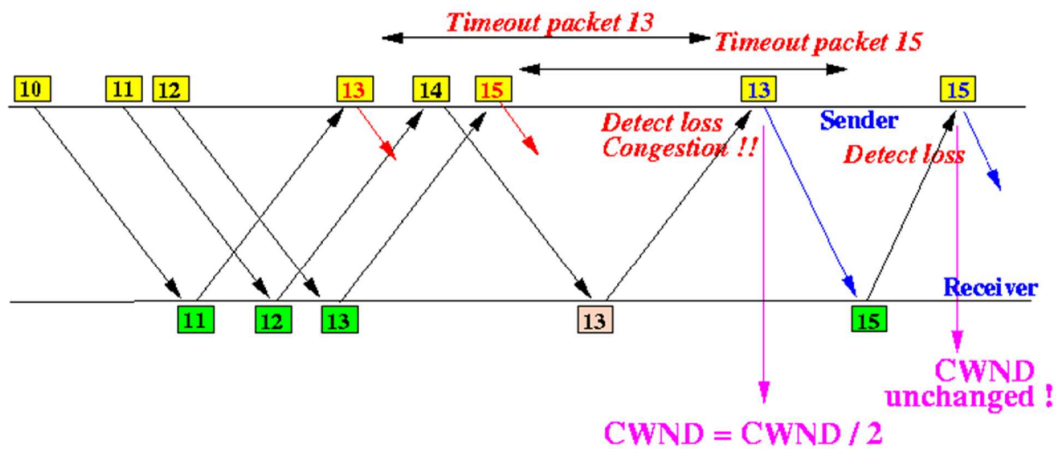
In aggiunta, un timer meno preciso potrebbe minimizzare la congestione, facendo entrare la connessione in modalità slow start.

Sistema di ritrasmissione

Per supportare il meccanismo di rilevazione della congestione, la stima dell'RTT di TCP Vegas deve essere molto precisa. Ogni volta che un segmento viene inviato e il relativo ACK ricevuto, la differenza fra i due istanti viene misurata e registrata.

Quando si riceve un ACK duplicato, il protocollo controlla se la differenza fra l'istante corrente e quello dell'invio del pacchetto corrispondente è maggiore del tempo di timeout. Se lo è, allora il messaggio viene ritrasmesso immediatamente, senza aspettare ulteriori ACK duplicati. In questo caso, la dimensione della finestra di congestione viene ridotta.

Se si ricevono degli ACK diversi dopo la ritrasmissione, viene effettuato un controllo per verificare se l'intervallo di tempo fra questi e il pacchetto che li ha prodotti è maggiore del tempo di timeout. Se lo è, il messaggio viene ritrasmesso immediatamente. Questo permette di gestire elegantemente anche eventuali altri pacchetti persi durante la prima ritrasmissione senza dover rallentare ulteriormente. Fintanto che le perdite si sono verificate prima della riduzione della cwnd, infatti, non vengono effettuate ulteriori riduzioni.



Confronto fra TCP Tahoe, Reno, NewReno e Vegas

TCP	3 ACK duplicati	Partial ACK	Full ACK
Tahoe	<i>Fast retransmit, entra in slow start</i>	$cwnd++$	$cwnd++$
Reno	<i>Fast retransmit, entra in fast recovery</i>	Esce da <i>fast recovery</i> , la finestra di pacchetti in viaggio viene ridotta, entra in <i>congestion avoidance</i>	Esce da <i>fast recovery</i> , la finestra di pacchetti in viaggio viene ridotta, entra in <i>congestion avoidance</i>
NewReno	<i>Fast retransmit, entra in fast recovery modificata</i>	<i>Fast retransmit</i> , la finestra di pacchetti in viaggio viene ridotta	Esce da <i>fast recovery</i> , la finestra di pacchetti in viaggio viene ridotta, entra in <i>congestion avoidance</i>
Vegas	Al primo ACK duplicato, <i>Fast retransmit</i>	Se il pacchetto del partial ACK è andato in timeout, <i>fast retransmit</i>	Continua in <i>congestion avoidance</i>

Selective ACKnowledgements (SACK)

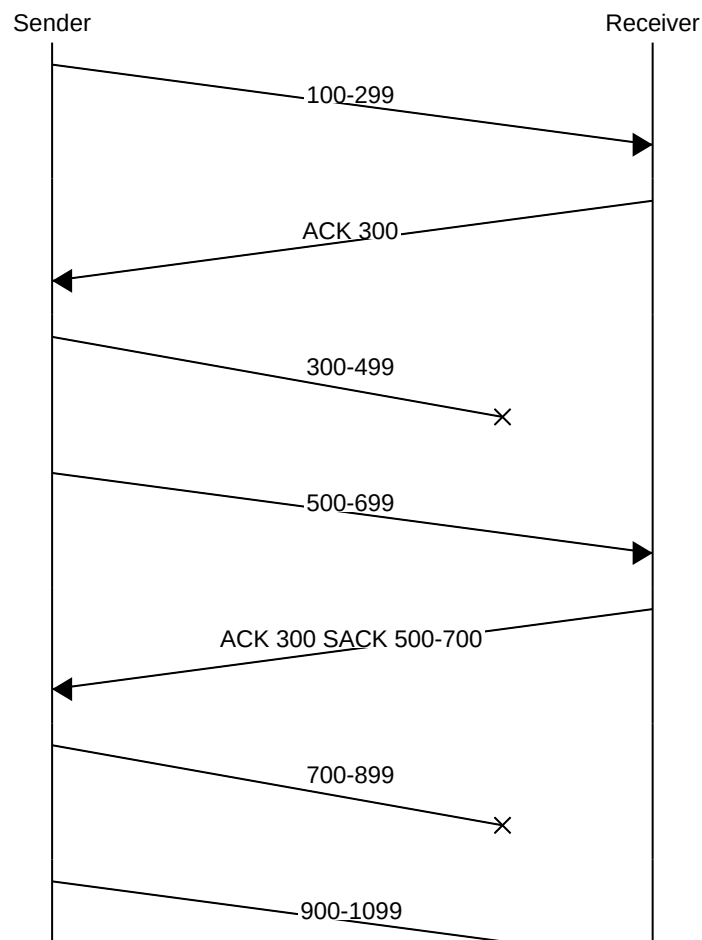
l'accettazione selettiva di TCP SACK fornisce un modo per risolvere i problemi legati alla congestione e alla caduta di più pacchetti.

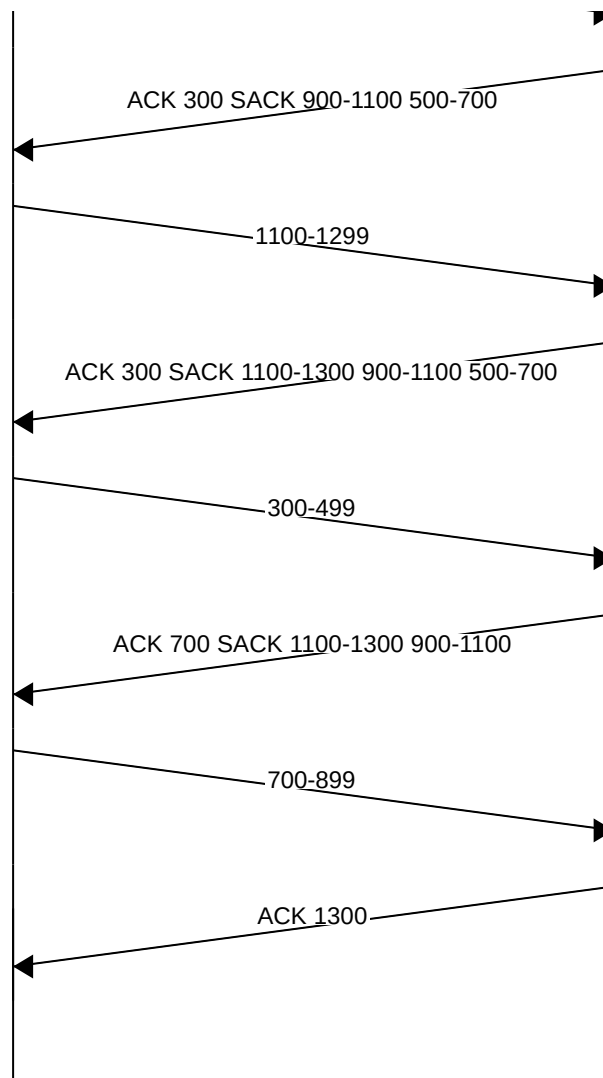
Senza SACK, l'applicazione ricevente accetterebbe solo l'ultimo pacchetto ricevuto in ordine. Nel caso si verifichi una perdita, tutti i pacchetti successivi a quello smarrito verrebbero scartati.

Con SACK, invece, il mittente deve solo ritrasmettere il pacchetto perso, perché gli altri saranno stati bufferizzati dal destinatario. Dato che si continua ad utilizzare un ACK cumulativo, si hanno delle informazioni piuttosto precise sulla ritrasmissione da effettuare.

Utilizzare la funzione SACK richiede abilitare due opzioni TCP. La prima, **SACK-Permitted**, deve essere flaggata nel pacchetto SYN, per esplicitare il supporto per la funzionalità. Inoltre, tutti i pacchetti che contengono dei SACK devono specificare l'omonima opzione.

SACK è compatibile con gli algoritmi di controllo della congestione presentati fin'ora. Anzi, al contrario, fornisce ulteriori informazioni utili per minimizzare i dati ritrasmessi inutilmente.





Data Reneging

Potrebbe capitare che il buffer per memorizzare i pacchetti in arrivo si riempia, e il destinatario sia costretto a scartare messaggi che aveva precedentemente confermato di possedere tramite SACK. In queste evenienze, è necessario che tutti i prossimi blocchi SACK riflettano il nuovo stato del buffer, informando il mittente dell'accaduto.

Proprio per questa inaffidabilità del meccanismo, il mittente deve mantenere comunque il classico meccanismo di timeout per trasmissione, e non considerare un messaggio confermato fino a quando non si riceve un ACK cumulativo che lo include.

TPC Westwood

Viene definito un sistema efficiente per reti ad alta velocità, sia cablate che wireless. Il miglioramento più significativo è osservabile nelle reti wireless con collegamenti lossy, cioè con la possibilità di perdere pacchetti.

Questo protocollo cerca di superare il dimezzamento netto della congestion window applicato da TCP Reno.

Questa stima è ottenuta applicando un filtro passa basso alla frequenza degli ACK di ritorno. L'incremento oltre la soglia è additivo, mentre il decremento è adattativo (AIAD).

BandWidth Estimate (BWE)

L'idea alla base di TCP Westwood è quella di trovare il valore migliore possibile a cui impostare la finestra di congestione nel momento in cui si registra della congestione.

Viene implementata una modifica dal lato del mittente, cercando di ottenere una stima della banda fra le due parti. Per ottenere un valore affidabile, TCP Westwood computa una **BandWidth Estimate (BWE)**, definita come il bottleneck della banda disponibile per la connessione.

Una prima stima può essere fatta tramite $b_k = \frac{d_k}{t_k - t_{k-1}}$. I campioni vengono poi filtrati tramite una media esponenziale mobile:

$$BE_k = \alpha BE_{k-1} + (1 - \alpha) \cdot \frac{b_k + b_{k-1}}{2}.$$

Dato che si vuole dare maggiore peso alle informazioni più recenti, anche il valore di α varia in funzione del tempo: $\alpha_k = \frac{2\tau - \Delta t_k}{2\tau + \Delta t_k}$, dove $\frac{1}{\tau}$ è la frequenza di taglio del filtro.

Gestione della perdita di un pacchetto

Quando dopo la ricezione di 3 ACK duplicati, il mittente resetta la finestra di congestione e la ssthresh proprio in base al valore indicato dalla BWE. Più precisamente

'I're ACK duplicati

$$ssthresh = BWE \cdot RTT_{min} / seg_size$$

$$cwnd = \min\{cwnd, ssthresh\}$$

Timeout

$$ssthresh = BWE \cdot RTT_{min} / seg_size$$

$$cwnd = 1$$

L'incremento di cwnd avviene seguendo le regole di TCP Reno.

Rispetto a Reno, Westwood reagisce meno brutalmente alle perdite sporadiche. Una piccola frazione di pacchetti persi casualmente non hanno un forte impatto sulla soglia della banda stimata. Per questo, generalmente il throughput è maggiore. Al contrario, nel caso di una perdita sostenuta, dovuta con ogni probabilità ad una forte congestione, l'arresto è anche più netto che in Reno.

Se due connessioni hanno lo stesso RTT, allora TCP Westwood assicura una buona fairness, per cui la banda del canale è divisa equamente fra i partecipanti.

Westwood è anche più "amichevole" rispetto ad altri protocolli, perché invece che provare ad incrementare costantemente la banda, cerca di trovare lo sweetspot che si avvicina alla situazione reale.

Grazie al successo, è implementato anche nel kernel Linux.

Esiste anche una piccola variante, chiamata Westwood+ in cui la stima viene fatta osservando l'intera finestra, quindi con un margine più ampio.

TCP Veno

Nome che nasce dalla fusione di Vegas e Reno. L'idea è quello di sfruttare il valore di N , che rappresenta il numero di pacchetti nel backlog per stabilire se la perdita osservata è una semplice perdita sporadica o è da interpretare come una situazione di congestione.

Nella fase di slow start, Venno si comporta esattamente come Reno, mentre nella fase di congestion avoidance viene applicata una versione modificata dell'algoritmo di Vegas. Lo stesso vale per la fase di fast recovery. Invece di effettuare un taglio netto della cwnd, in base al valore di N tale riduzione può essere ridotta ad $\frac{1}{5}$.

```

Additive_Increase():
    N = cwnd * ( 1 - RTT_min / RTT )
    if N < beta:
        cwnd += 1/cwnd /* la banda è sottoutilizzata */
    else if N >= beta and sequence_number % 2 == 0: /* la banda è ben
utilizzata, si incrementa una volta sì, una no */
        cwnd += 1/cwnd

Decrease():
    if N < beta: /* probabile perdita sporadica */
        ssthresh = cwnd * 4 / 5
    else: /* probabile congestione */
        ssthresh = cwnd / 2
    cwnd = ssthresh

```

In altre parole Venet rivede i sistemi di **Additive Increase, Multiplicative Decrease (AIMD)**. Tutto il resto è preso direttamente da Reno. Questo include la slow start, la fast retransmit, il fast recovery etc.

TPC BIC

Le connessioni odierne raggiungono bande molto grandi, che si prestano bene all'aumento esponenziale, ma diventano problematiche nel momento in cui l'aumento diventa lineare.

Per venire incontro alle nuove caratteristiche della rete, si è provato a modificare i parametri utilizzati da Reno. Nello specifico, l'incremento rimane lineare, ma si muove con un incremento di 32, mentre il dimezzamento diventa una riduzione di $\frac{1}{8}$ rispetto alla soglia originale. Questa scelta permette di migliorare parecchio l'utilizzo del canale, ma a discapito degli altri utenti della banda, con i quali non convive bene.

L'approccio di BIC è di cercare di effettuare una crescita binaria. Dati un valore di w_{max} , pari al massimo teorico che il canale è in grado di supportare e w_{min} che è la velocità attuale, l'incremento sarà tanto maggiore quanto più grande è la distanza fra questi due valori. Il valore dell'incremento è comunque compreso fra due valori scelti in precedenza di s_{min} , s_{max} .

Proprio perché il valore minimo di incremento è fissato, anche dopo aver raggiunto il limite massimo, la soglia continua ad essere incrementata.

A differenza di BIC, dopo aver superato il valore di w_{max} , CUBIC determina di essere in una connessione con una capacità molto elevata, e passa ad un incremento esponenziale.